Priority Queue

Priority Queue

Collections: Insert and delete items. Which item to delete

Stack: Remove the item most recently added

Queue: Remove the item least recently added

Randomized queue: Remove a random item

Priority queue: Remove the largest (or smallest) item

Priority Queue

Operation	Argument	Return value
insert	P	
Insert	Q	
Insert	E	
remove max		Q
Insert	X	
Insert	A	
Insert	M	
remove max		X
Insert	P	
Insert	L	
Insert	E	
remove max		P

Priority Queue API:

```
class maxPQ():
    def __init__(self):     # Create an empty priority queue
    def __init__(self, q):  # create a priority queue with keys
    def insert(self, key):  # insert a key into the priority queue
    def delMax(self):       # return and remove the largest key
    def IsEmpty(self):      # is the priority queue empty?
    def max(self):          # return the largest key
    def size(self):         # number of entries in the priority queue
```

Priority queue applications:

- Event-driven simulation [customer is a line, colliding particle]
- Numerical computation [reduce roundoff error]
- Data compression [Huffman codes]
- Graph searching [Dijkstra's algorithm, Prim's algorithm]
- Number theory [sum of powers]
- Artificial Intelligence [A8 search]
- Statistics [maintain largest M value in sequence]
- Operating systems [load balancing, interrupt handling]
- Discrete optimization [bin packing, scheduling]
- Spam filtering [Bayesian spam filter]

Priority queue client example

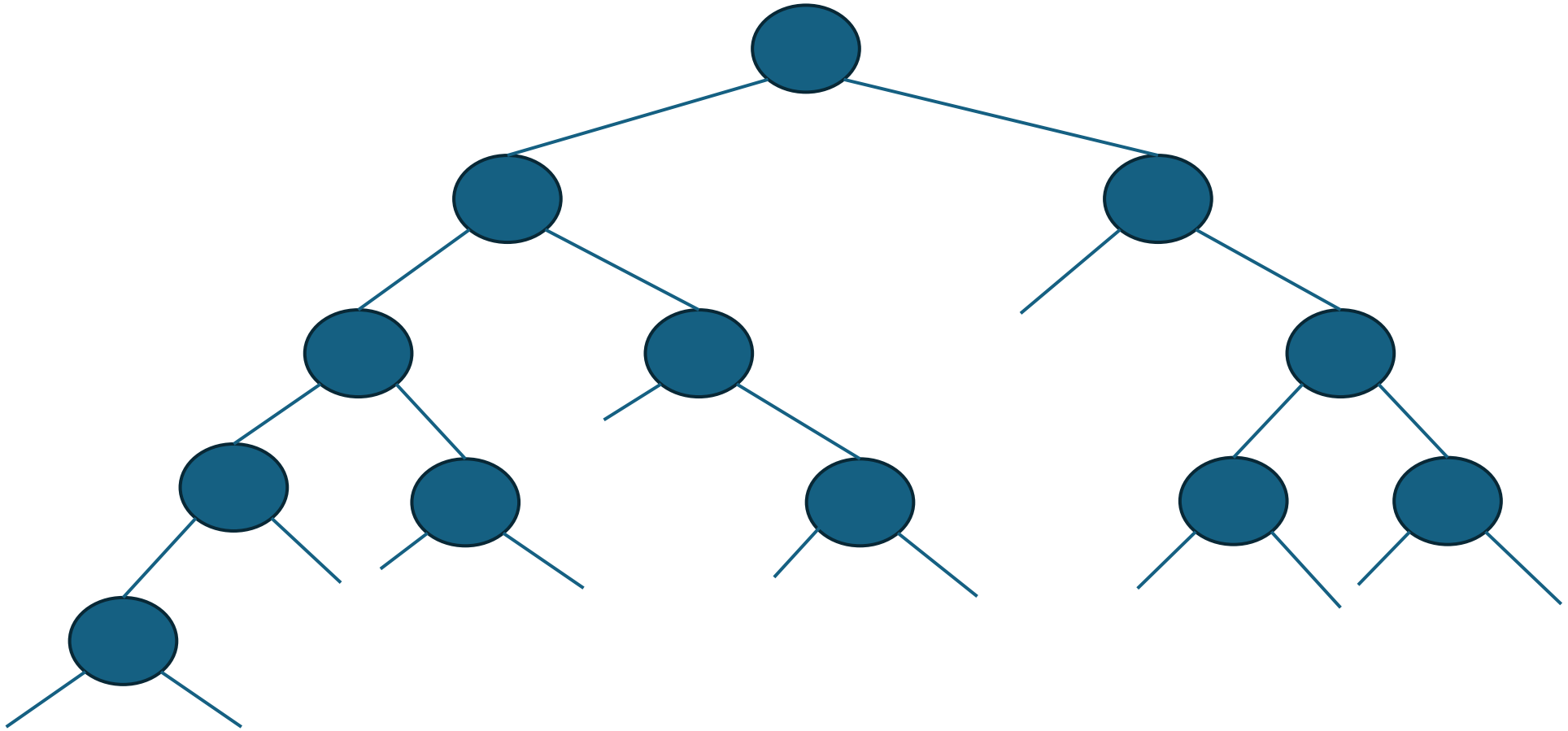
Challenge. Find the largest M items in a stream of N items.

order of growth of finding the largest M in a stream of N items

implementation	time	space
sort	N log N	N
elementary PQ	M N	M
binary heap	N log M	M
best in theory	N	M

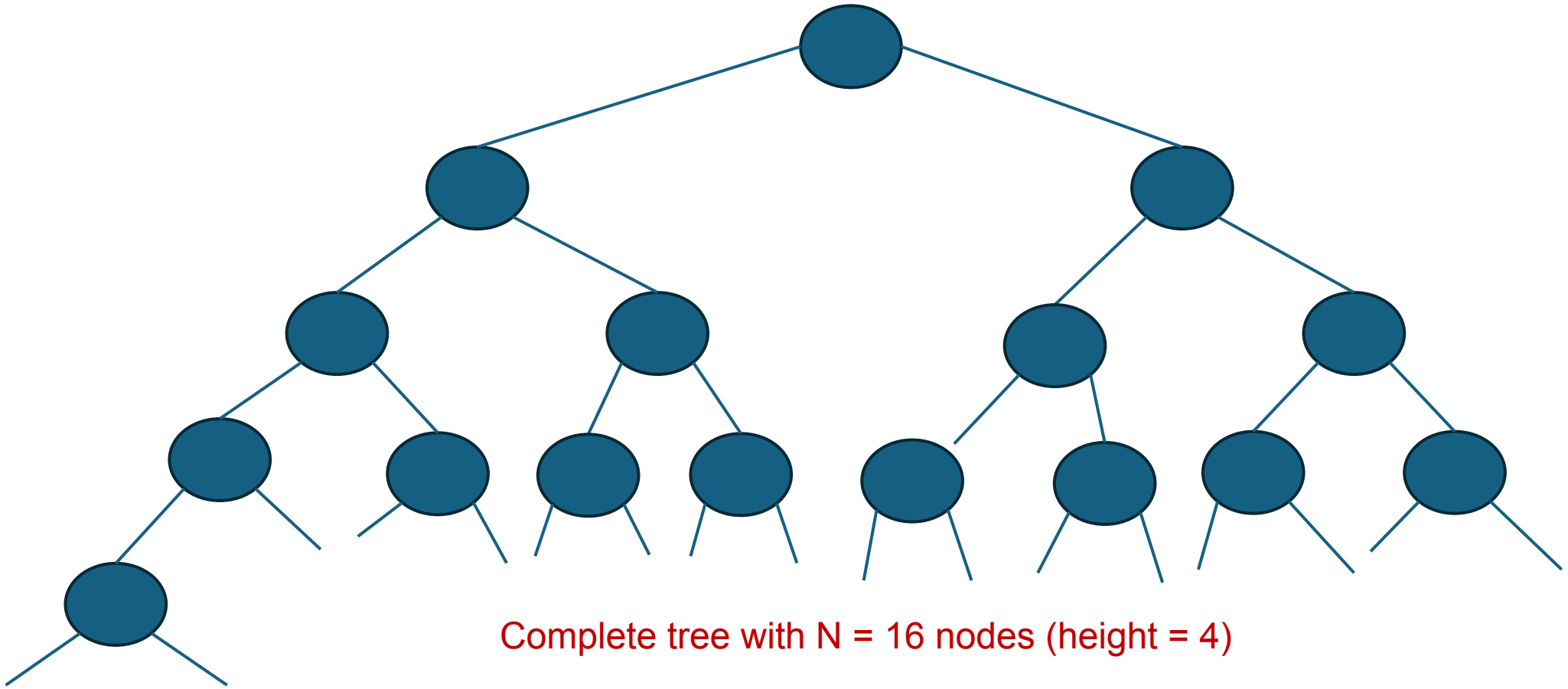
Complete binary tree:

Binary tree: Empty or node with links to left and right binary trees



Complete binary tree:

Complete tree: Perfect balanced, except for bottom level



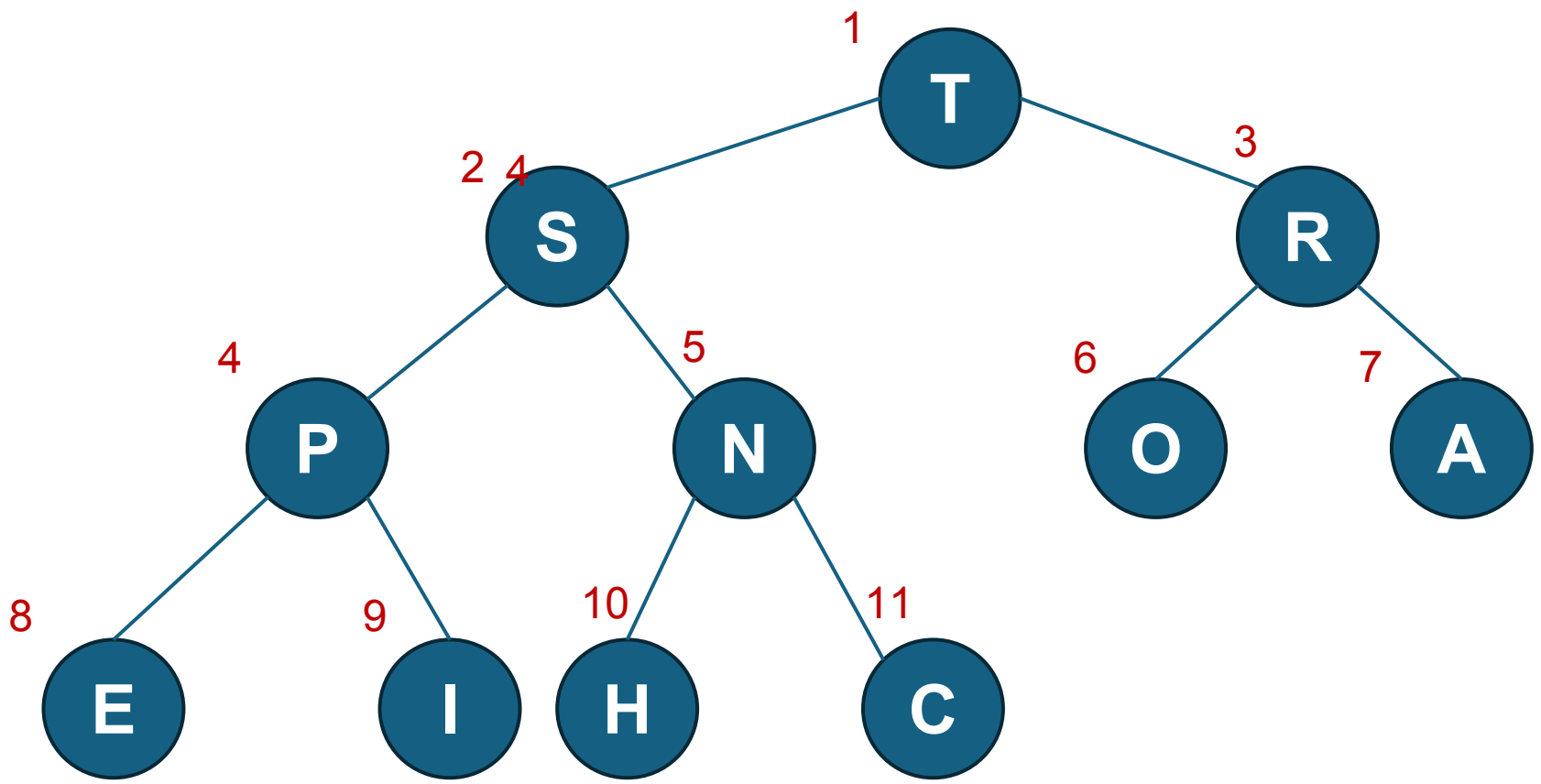
Complete tree with N = 16 nodes (height = 4)

Observation:

- Property: Height of complete tree with N node is $\lfloor \log N \rfloor$
- Height of the tree only increases when N is a power of 2

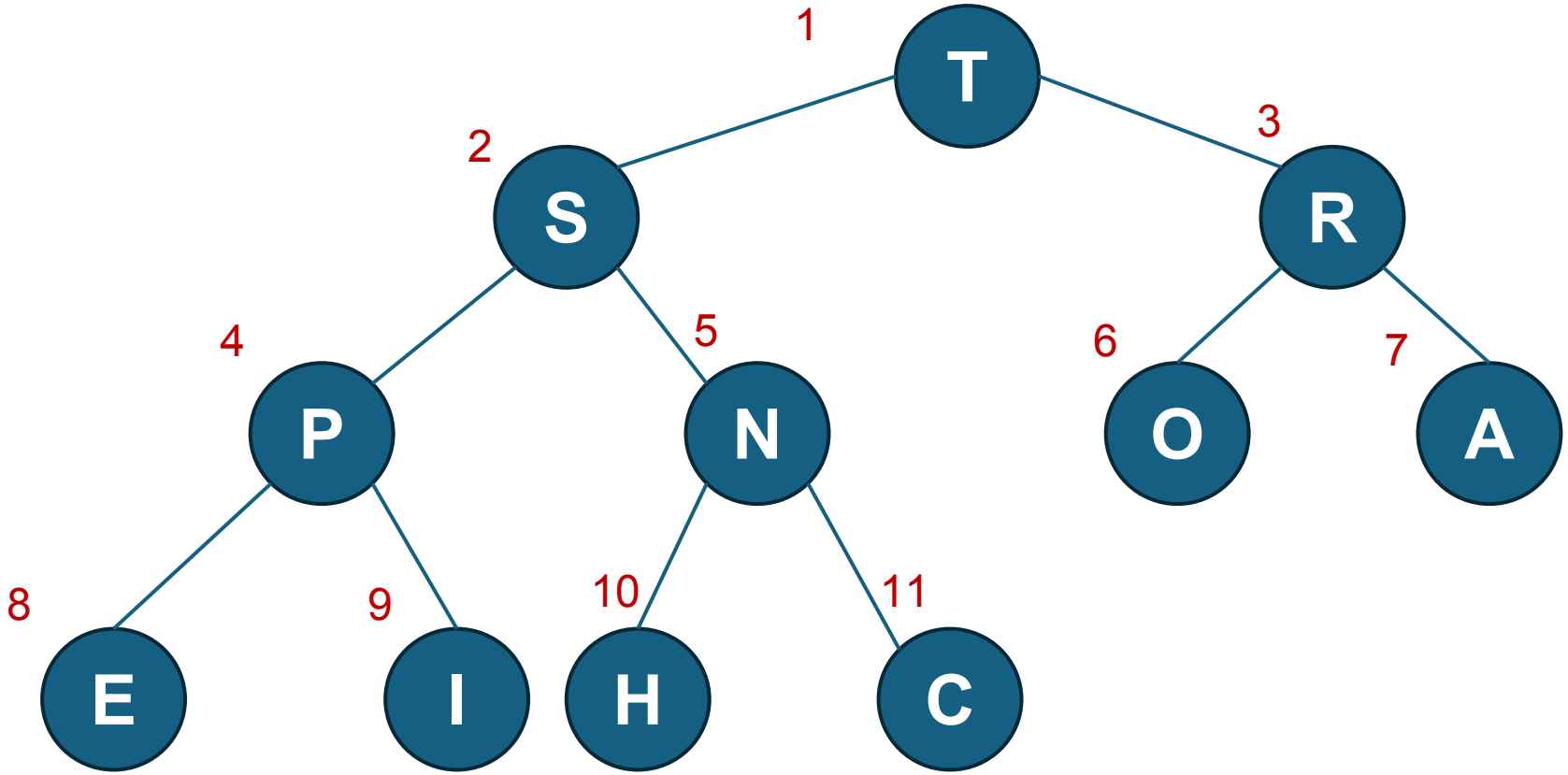
Binary heap representations

- Binary heap: Array representation of a heap-ordered complete binary tree
- Heap-ordered binary tree:
  - Keys in node
  - Parent’s key no smaller than children’s keys



Heap representation

i		0	1	2	3	4	5	6	7	8	9	10	11
a[i]		-	T	S	R	P	N	O	A	E	I	H	C



Heap representation

Array representation:

- Indices start at 1
- Takes nodes in level order
- No explicit links needed!

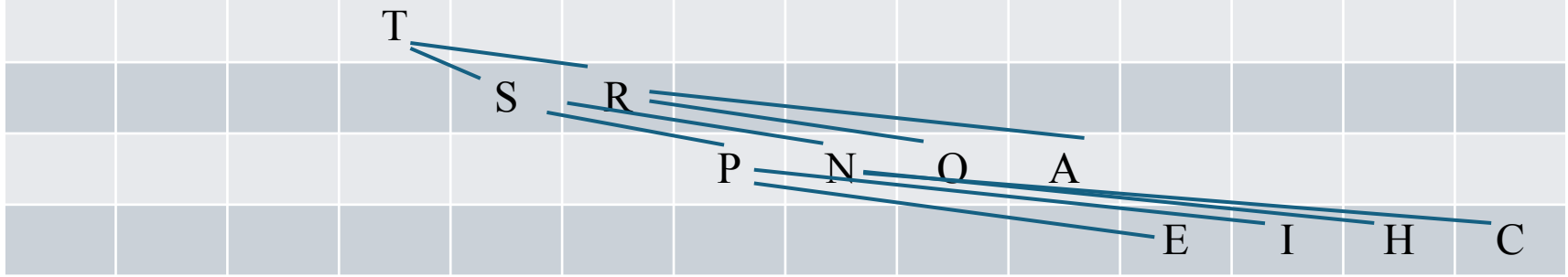
Binary heap properties:

Proposition: largest key is a[1], which is root of binary tree

Proposition: Can use array indices to move through tree

- Parent of node at k is at k//2
- Children of node at k are at 2k and 2k+1

i		0	1	2	3	4	5	6	7	8	9	10	11
a[i]		-	T	S	R	P	N	O	A	E	I	H	C



QUIZ

Which of the following arrays does not represent a max-oriented binary heap?

1) 19 18 17 16 15 14 13 12 11 10
2) 10 10 10 10 10 10 10 10 10 10
3) 34 30 29 27 25 17 16 19 22 24
4) 30 27 23 17 16 15 13 14 18 11

QUIZ

Which of the following arrays does not represent a max-oriented binary heap?

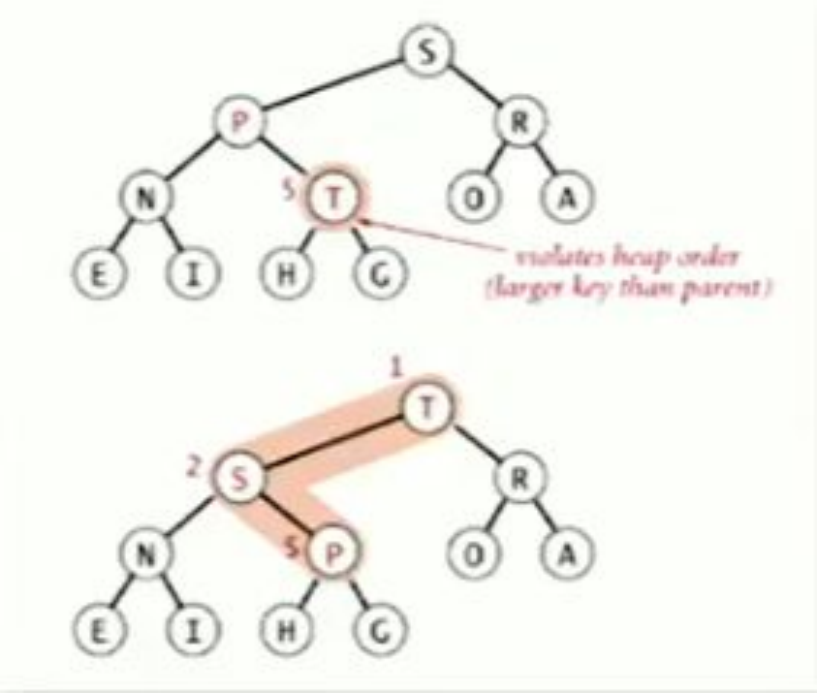
1) 19 18 17 16 15 14 13 12 11 10
2) 10 10 10 10 10 10 10 10 10 10
3) 34 30 29 27 25 17 16 19 22 24
4) 30 27 23 17 16 15 13 14 18 11

4 - In the corresponding binary tree, 17 is a parent of 18, which violates the heap-order property.

Promotion in a heap

Scenario: Child's key becomes larger kay than its parent's key

- To eliminate the violation:
  - Exchange key in child with key in parent
  - Repeat until head order restore



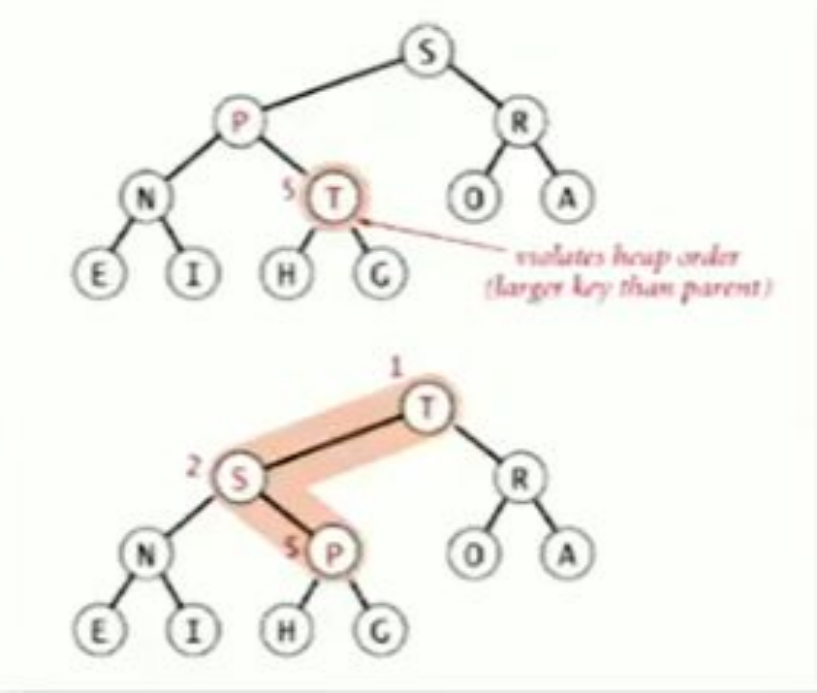
Peter principle: Node promoted to level of incompetence

Promotion in a heap

Scenario: Child's key becomes larger kay than its parent's key

- To eliminate the violation:
  - Exchange key in child with key in parent
  - Repeat until head order restore

```
def swim(k):
    while k > 1 and pq[k//2] < pq[k]:
        a[k], a[k//2] = a[k//2], a[k]
        k = k//2
```



Peter principle: Node promoted to level of incompetence

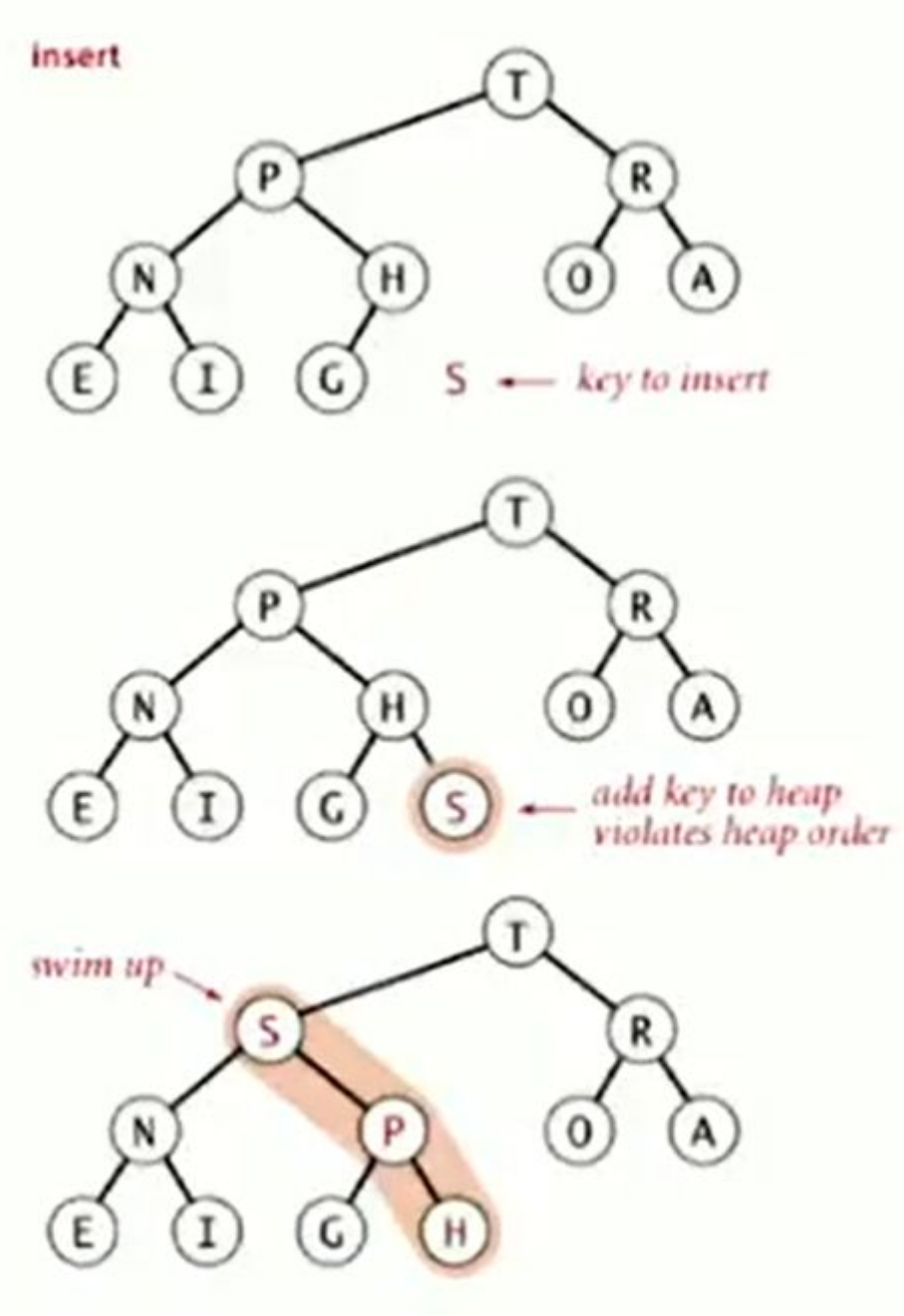
Insertion in a heap

Insert: Add node at end on a heap pq,

then swim it up

Cost: At most 1 + log N compares

```
def insert(x):
    pq[N+1], N = x, N+1
    swim(N)
```

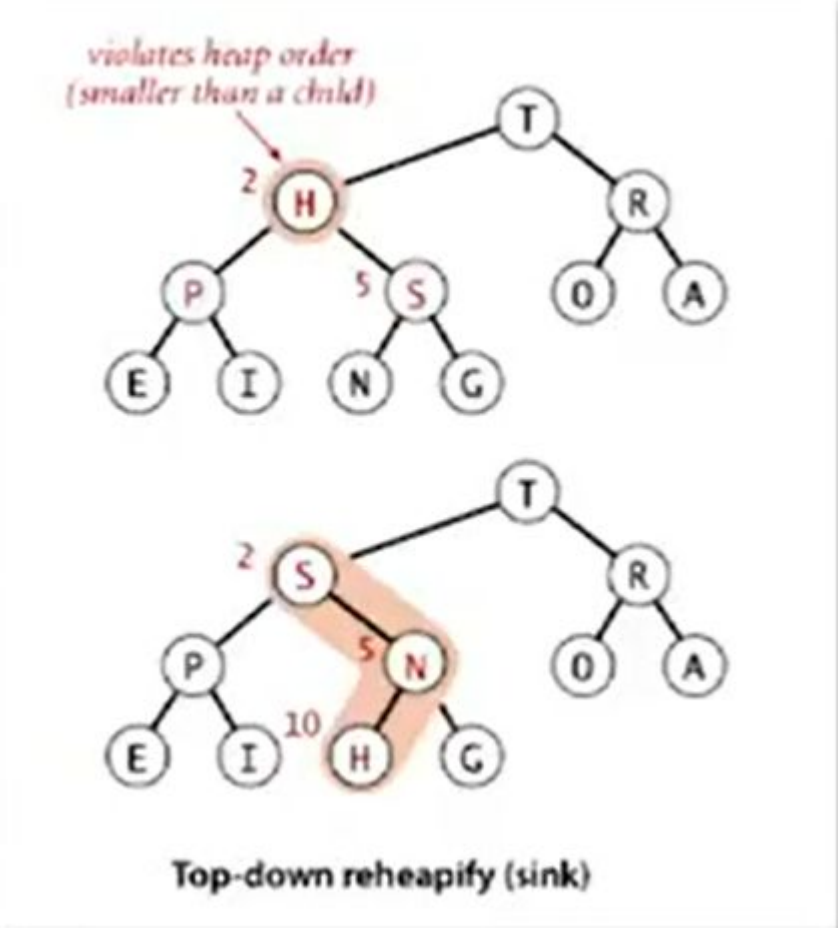


Demotion in a heap

Scenario: Parent's key become smaller than one (or both) of its children's

To eliminate the violation:

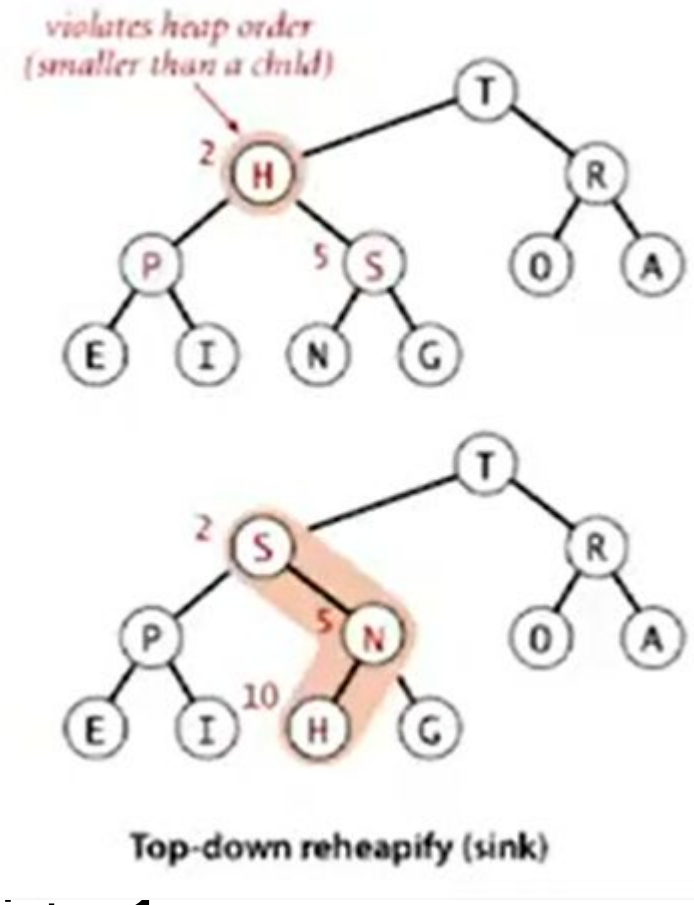
- Exchange key in parent with key in larger chi
- Repeat until heap order restore



Top-down reheapify (sink)

Power struggle: better subordinate promoted

Demotion in a heap



Top-down reheapify (sink)

To eliminate the violation:

- Exchange key in parent with key in larger child
- Repeat until heap order restore

```
def sink(parent):
    while 2*parent < N:
        bigsibling = 2*parent
        if bigsibling < N and a[bigsibling] < a[bigsibling+1]: j += 1
        if a[parent] >= a[bigsibling]: break
        a[parent], a[nigsibling] = a[bigsibling], a[parent]
        parent = bigsibling
```

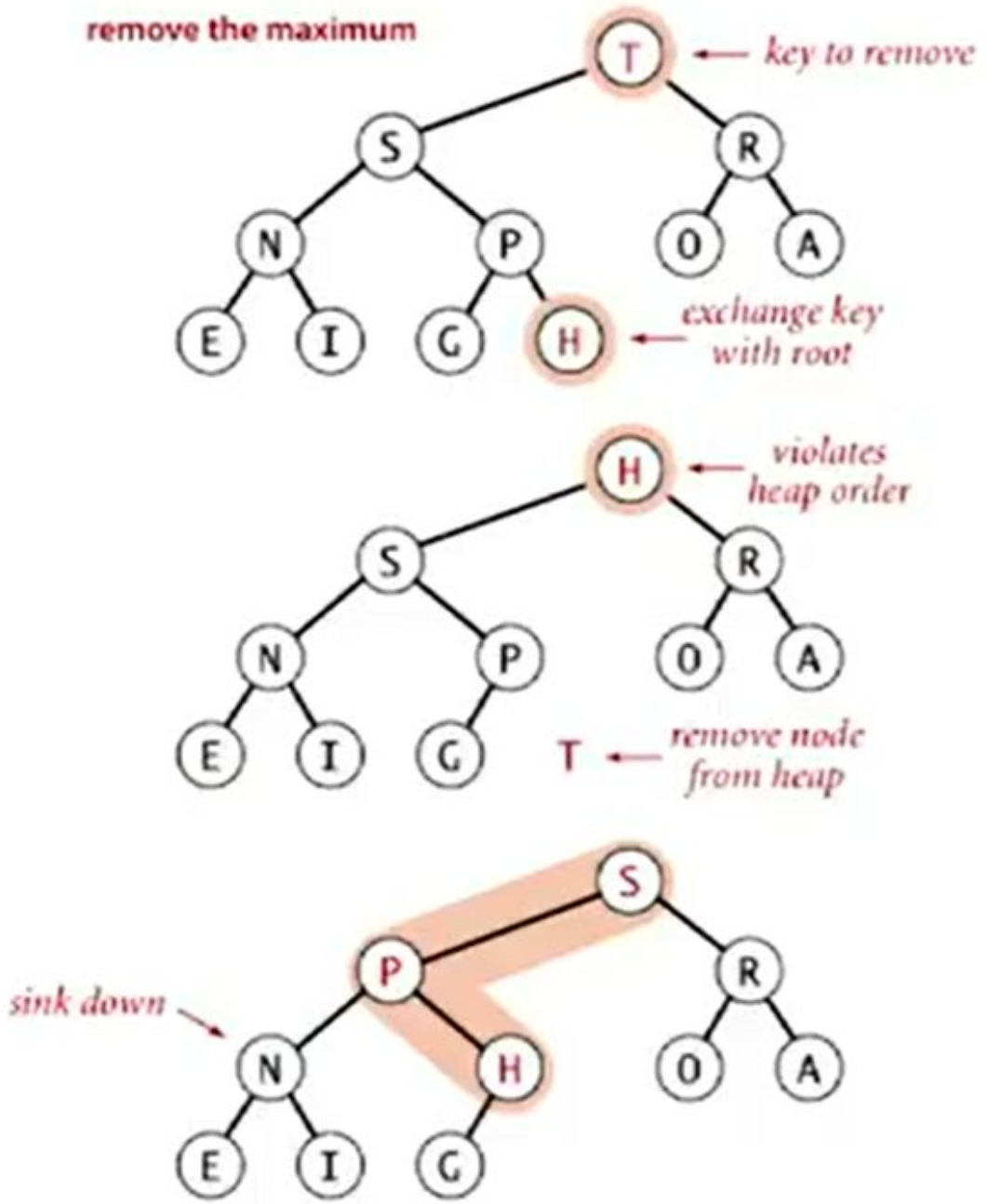
Power struggle: better subordinate promoted

Delete the maximum

Delete max: Exchange root with node at end, then sink it down

Cost: At most 2 log N compares

```
def delMax():
    max = pq[1]
    a[1], a[N] = a[N], a[1]
    N -= 1
    sink(1)
    pq[N+1] = None
    return max
```

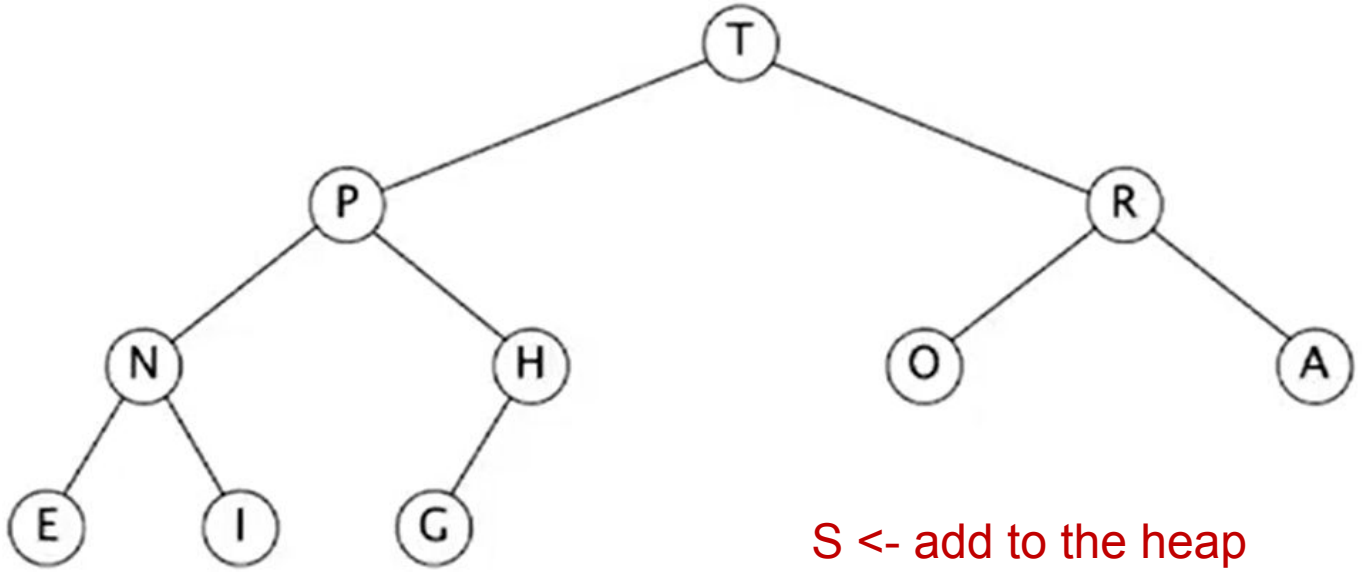


Binary heap demo

Insert: Add node at the end, then swim it up

Remove the maximum: Exchange root with node at end, then sink it down

insert S



S <- add to the heap

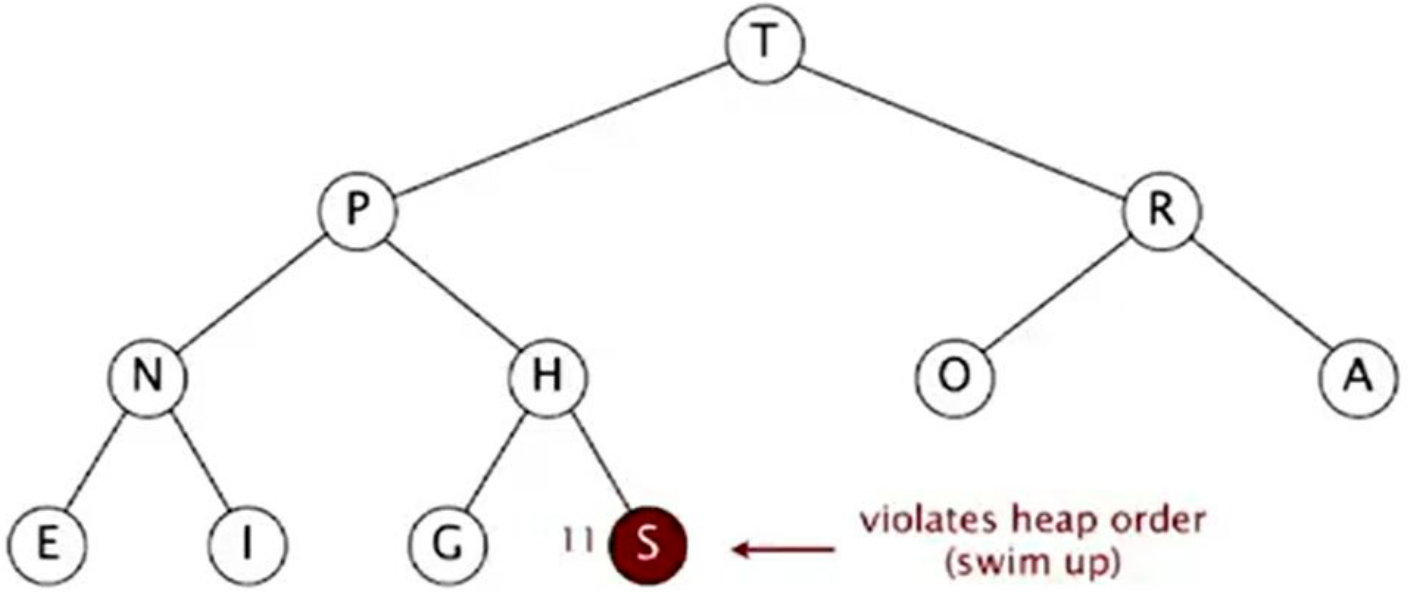
T	P	R	N	H	O	A	E	I	G	

Binary heap demo

Insert: Add node at the end, then swim it up

Remove the maximum: Exchange root with node at end, then sink it down

insert S



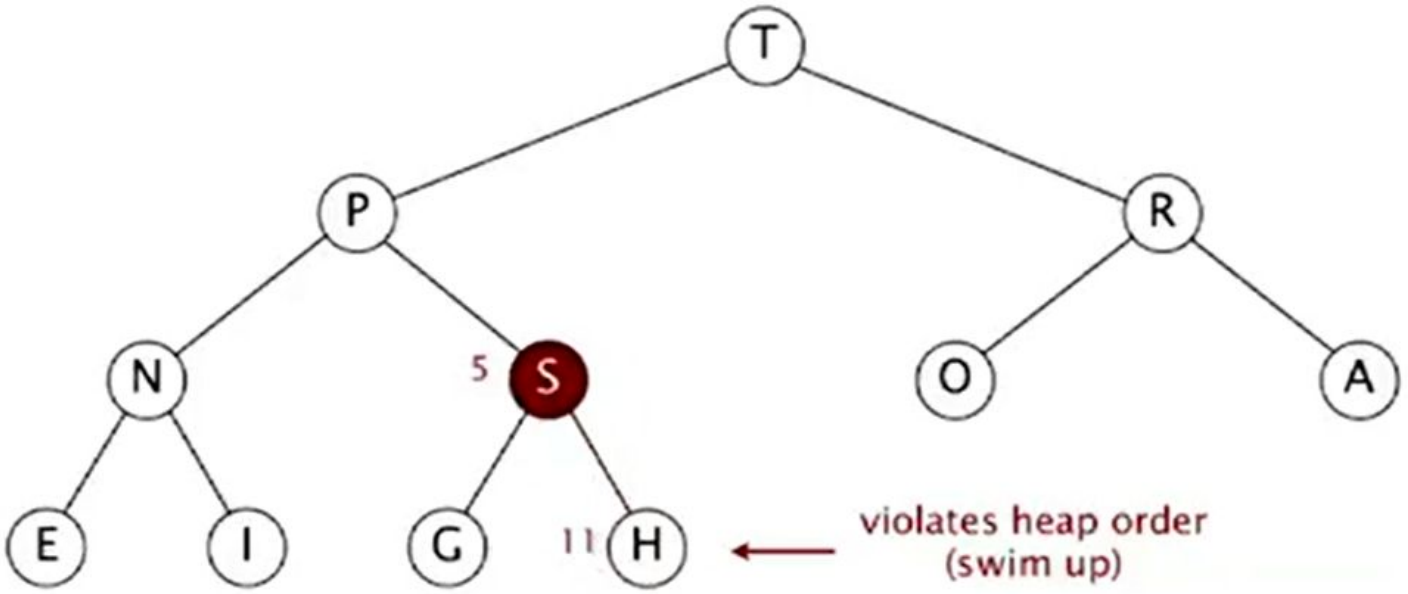
T	P	R	N	H	O	A	E	I	G	S

Binary heap demo

Insert: Add node at the end, then swim it up

Remove the maximum: Exchange root with node at end, then sink it down

insert S



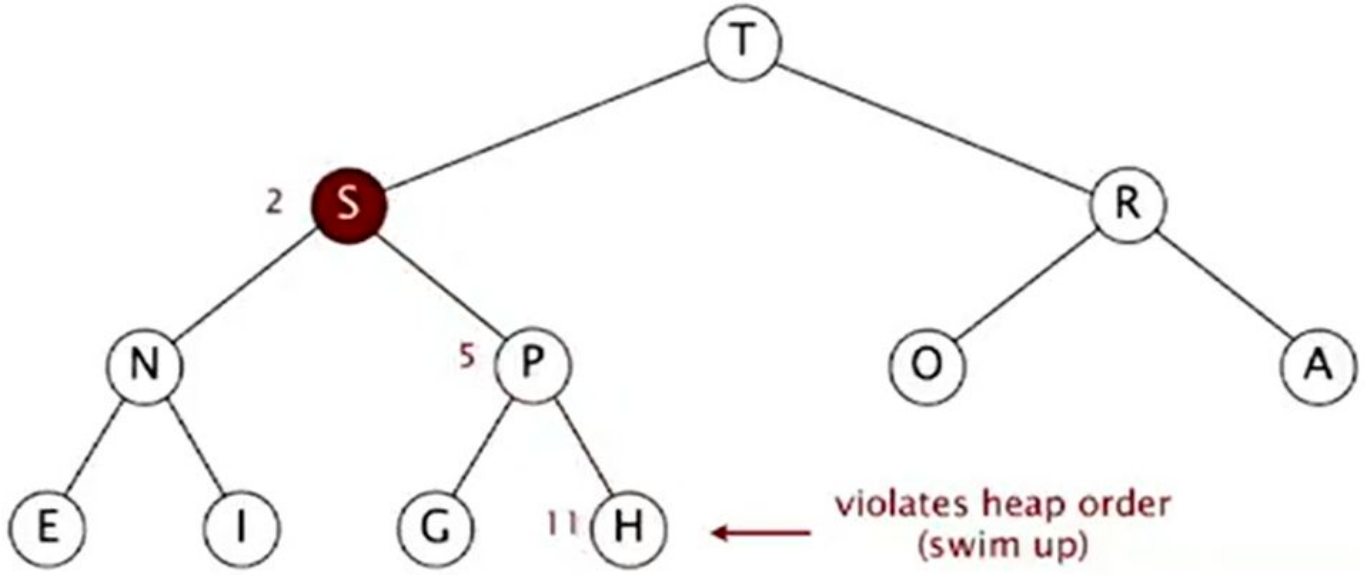
T	P	R	N	S	O	A	E	I	G	H

Binary heap demo

Insert: Add node at the end, then swim it up

Remove the maximum: Exchange root with node at end, then sink it down

insert S



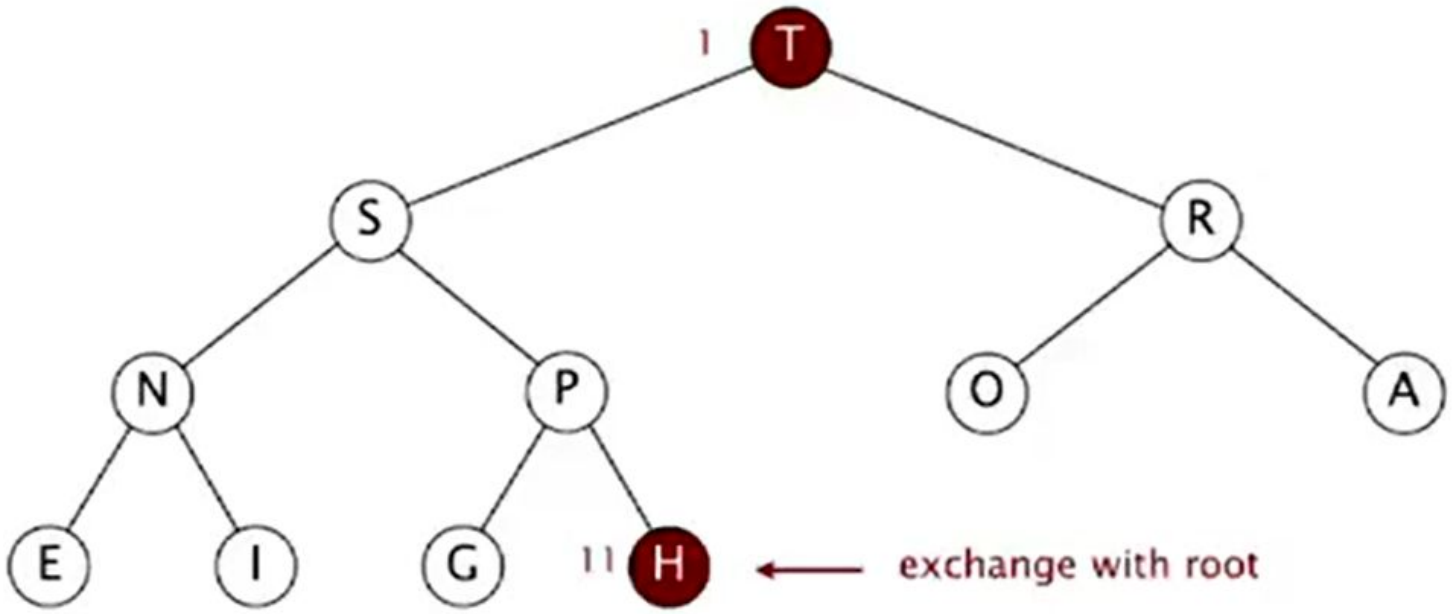
T	S	R	N	P	O	A	E	I	G	H

Binary heap demo

Insert: Add node at the end, then swim it up

Remove the maximum: Exchange root with node at end, then sink it down

remove max



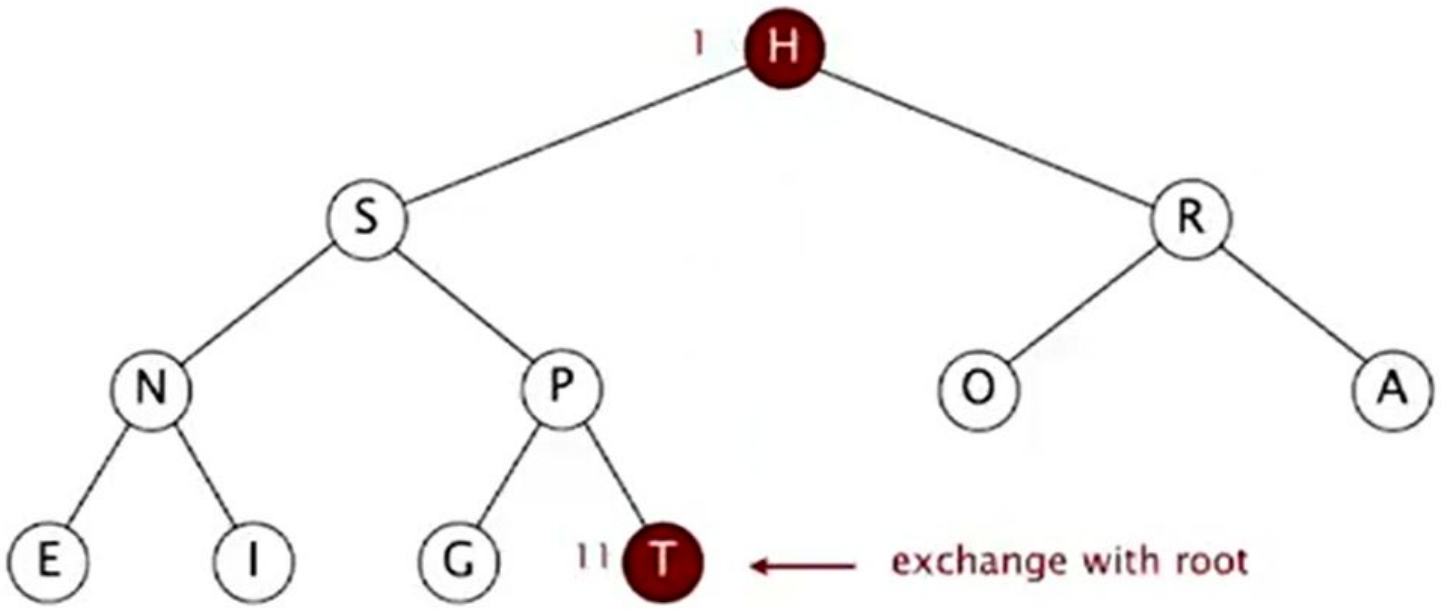
T	S	R	N	P	O	A	E	I	G	H

Binary heap demo

Insert: Add node at the end, then swim it up

Remove the maximum: Exchange root with node at end, then sink it down

remove max



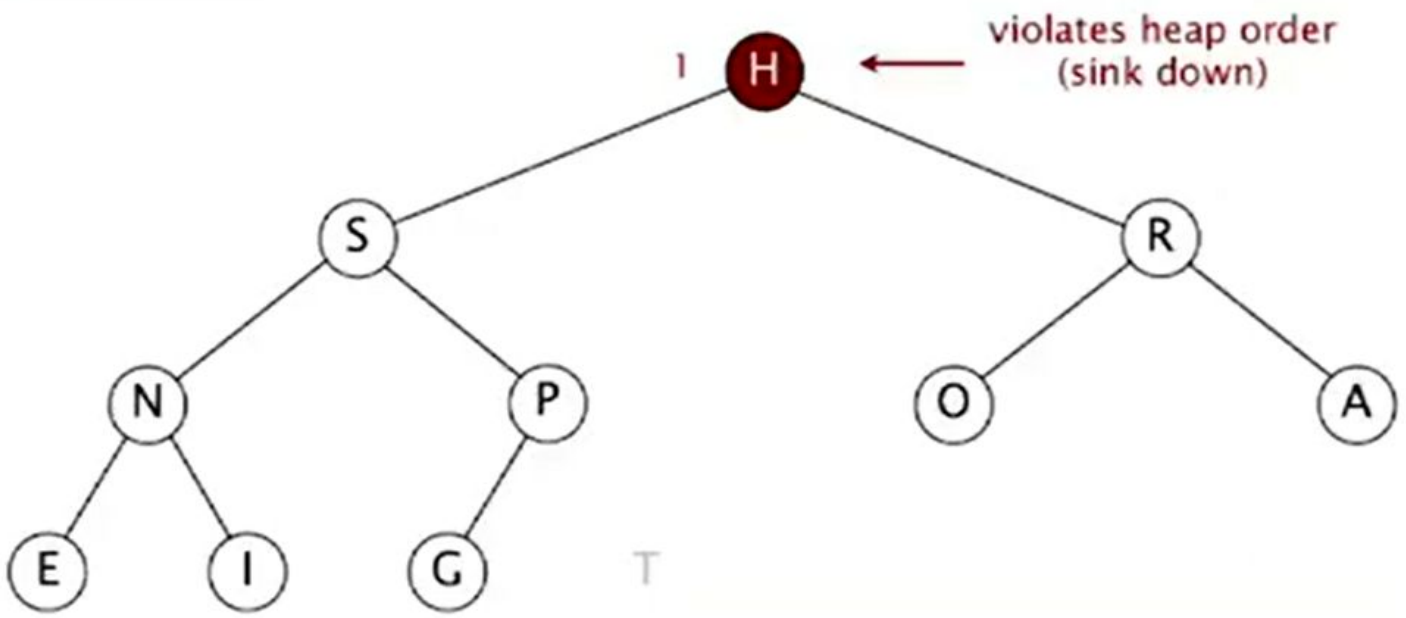
H	S	R	N	P	O	A	E	I	G	T

Binary heap demo

Insert: Add node at the end, then swim it up

Remove the maximum: Exchange root with node at end, then sink it down

remove max



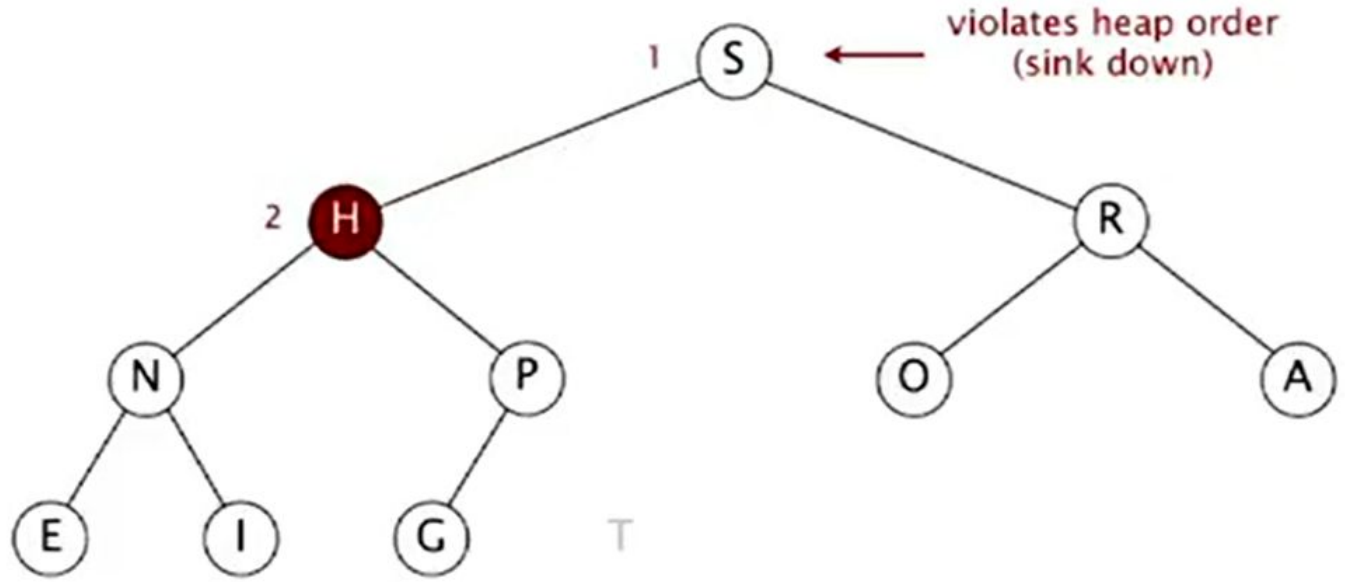
H	S	R	N	P	O	A	E	I	G	T

Binary heap demo

Insert: Add node at the end, then swim it up

Remove the maximum: Exchange root with node at end, then sink it down

remove max



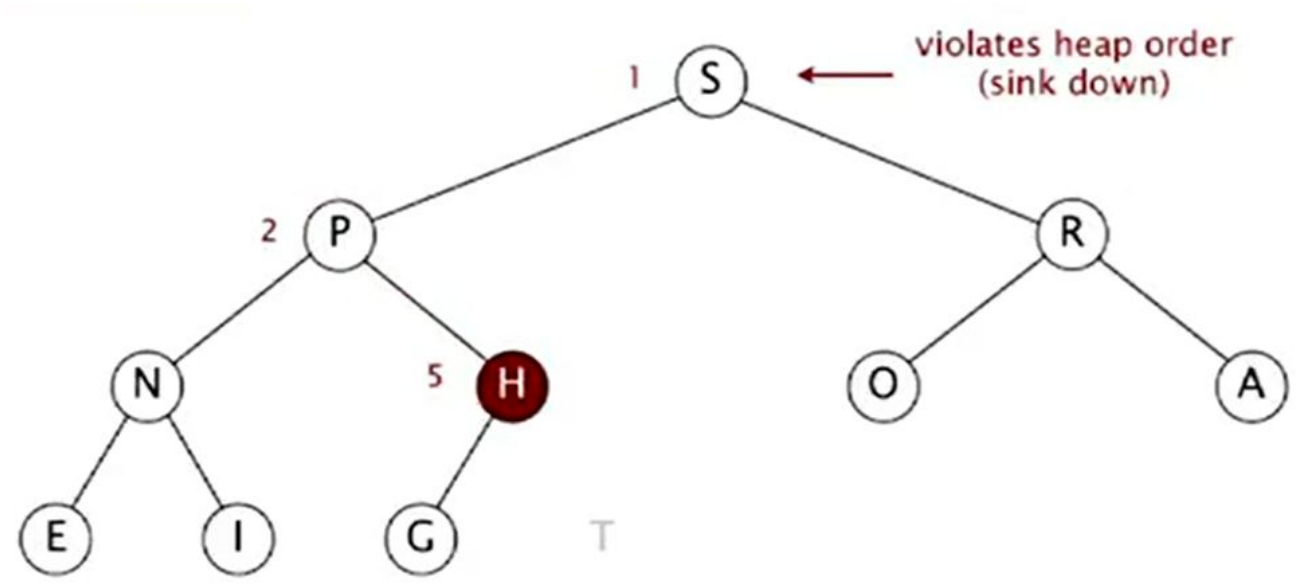
S	H	R	N	P	O	A	E	I	G	T

Binary heap demo

Insert: Add node at the end, then swim it up

Remove the maximum: Exchange root with node at end, then sink it down

remove max



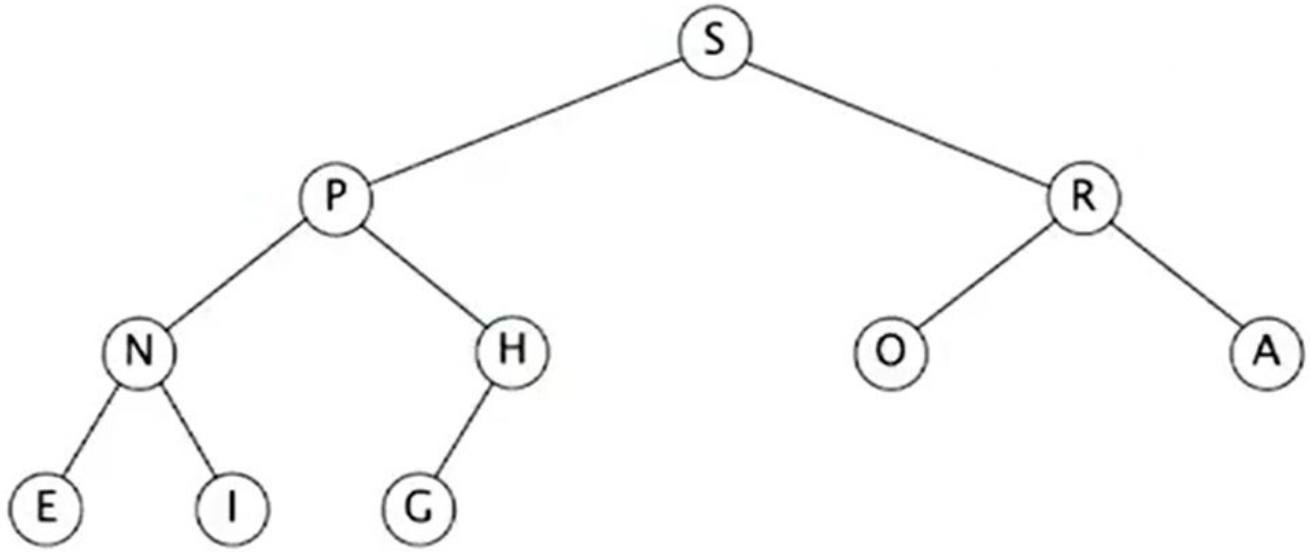
S	P	R	N	H	O	A	E	I	G	T

Binary heap demo

Insert: Add node at the end, then swim it up

Remove the maximum: Exchange root with node at end, then sink it down

heap ordered



S	P	R	N	H	O	A	E	I	G	T

Priority queues implementation cost summary:

order-of-growth of running time for priority queue with N items

implementation	insert	del max	max
unordered array	1	N	N
ordered array	N	1	1
binary heap	log N	log N	1
d-ary heap	$\log_d N$	$d \log_d N$	1
Fibonacci	1	log N †	1
impossible	1	1	1

← why impossible?

† amortized

QUIZ:

Suppose that an array a[] is a max-heap that contains the distinct integer keys 1, 2, . . . , N with N ≥ 7. The key N must be in a[1] and the key N-1 must be in either a[2] or a[3]. Where must the key N-2 be?

1) 2 or 3
2) 4, 5, 6, or 7
3) 2, 3, 4, 5, 6, or 7
4) 1, 2, 3, 4, 5, 6, or 7

QUIZ:

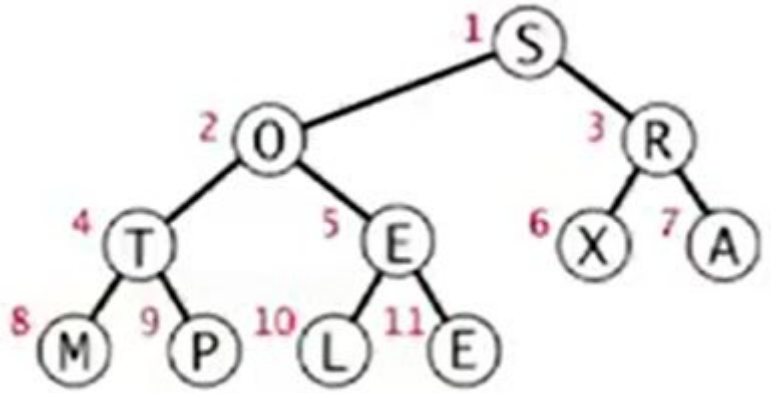
Suppose that an array a[] is a max-heap that contains the distinct integer keys 1, 2, . . . , N with N ≥ 7. The key N must be in a[1] and the key N-1 must be in either a[2] or a[3]. Where must the key N-2 be?

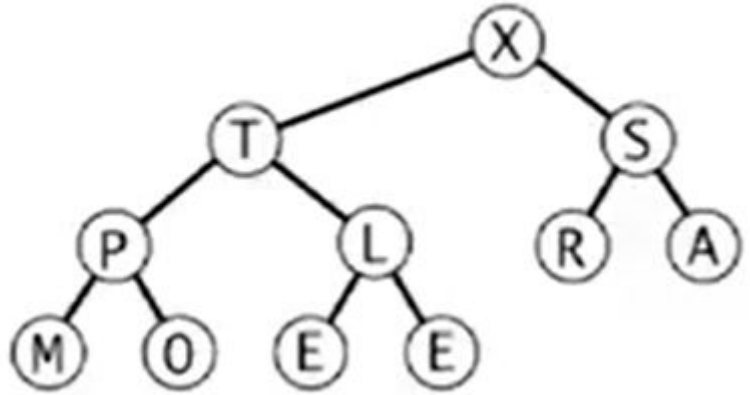
1) 2 or 3
2) 4, 5, 6, or 7
3) 2, 3, 4, 5, 6, or 7
4) 1, 2, 3, 4, 5, 6, or 7

3)

Heapsort

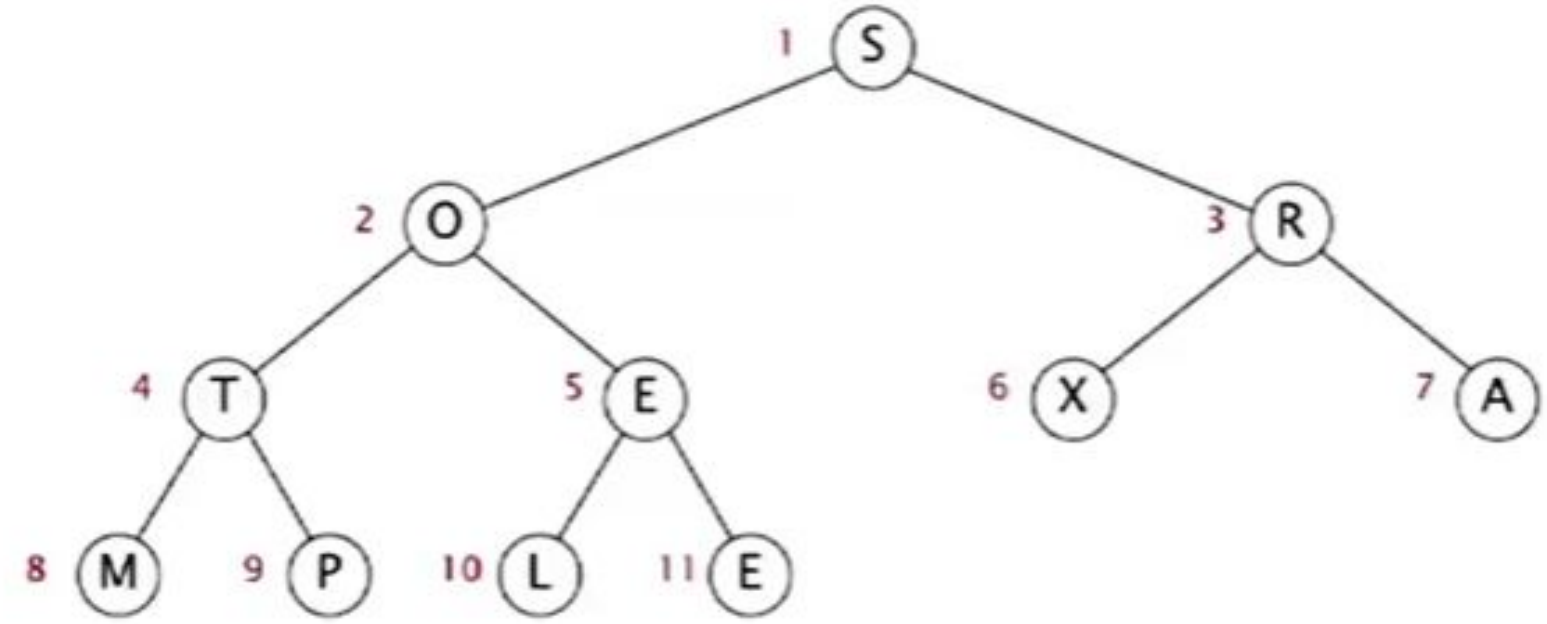
- Basic plan for in-place sort
- Create max-heap (or min-heap) with all N keys
- Repeatedly remove the maximum kay
  - Start with array of kays in arbitrary order
  - Build a max-heap (in-place
  - Sorted result (in-place)





Heapsort demo

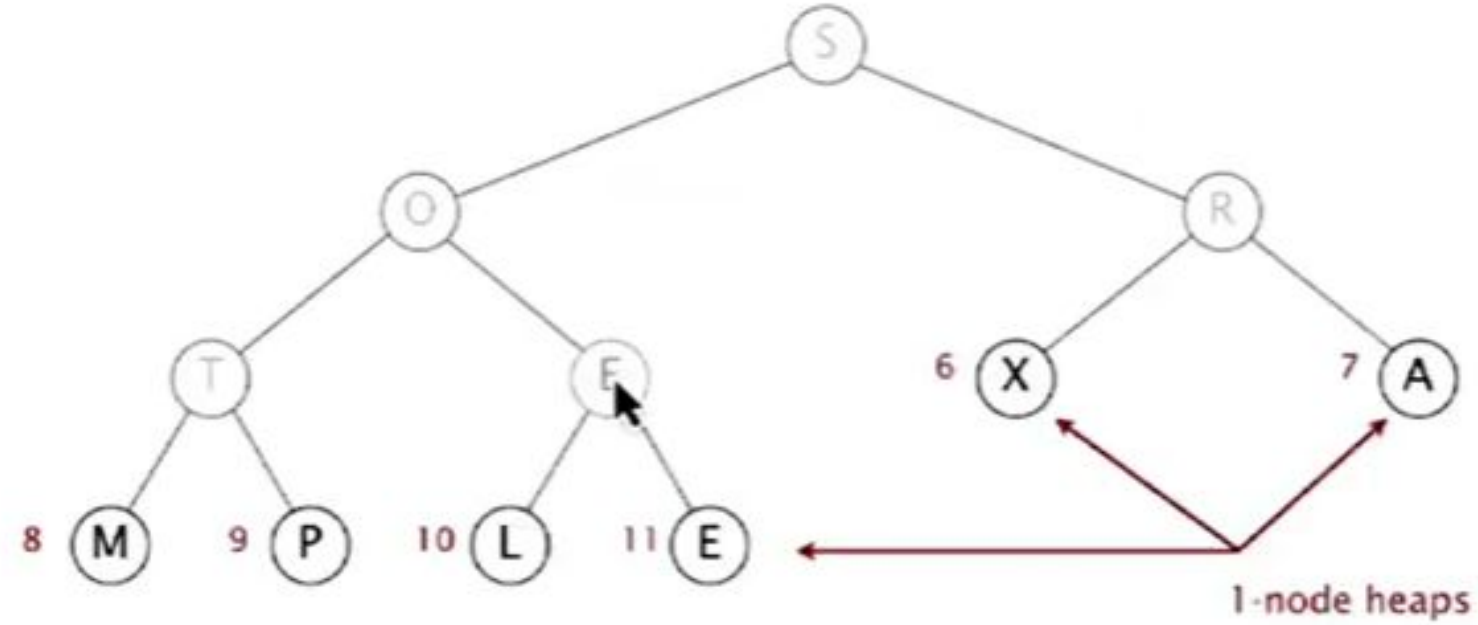
- Heap construction – build max heap using bottom-up method:



S	O	R	T	E	X	A	M	P	L	E

Heapsort demo

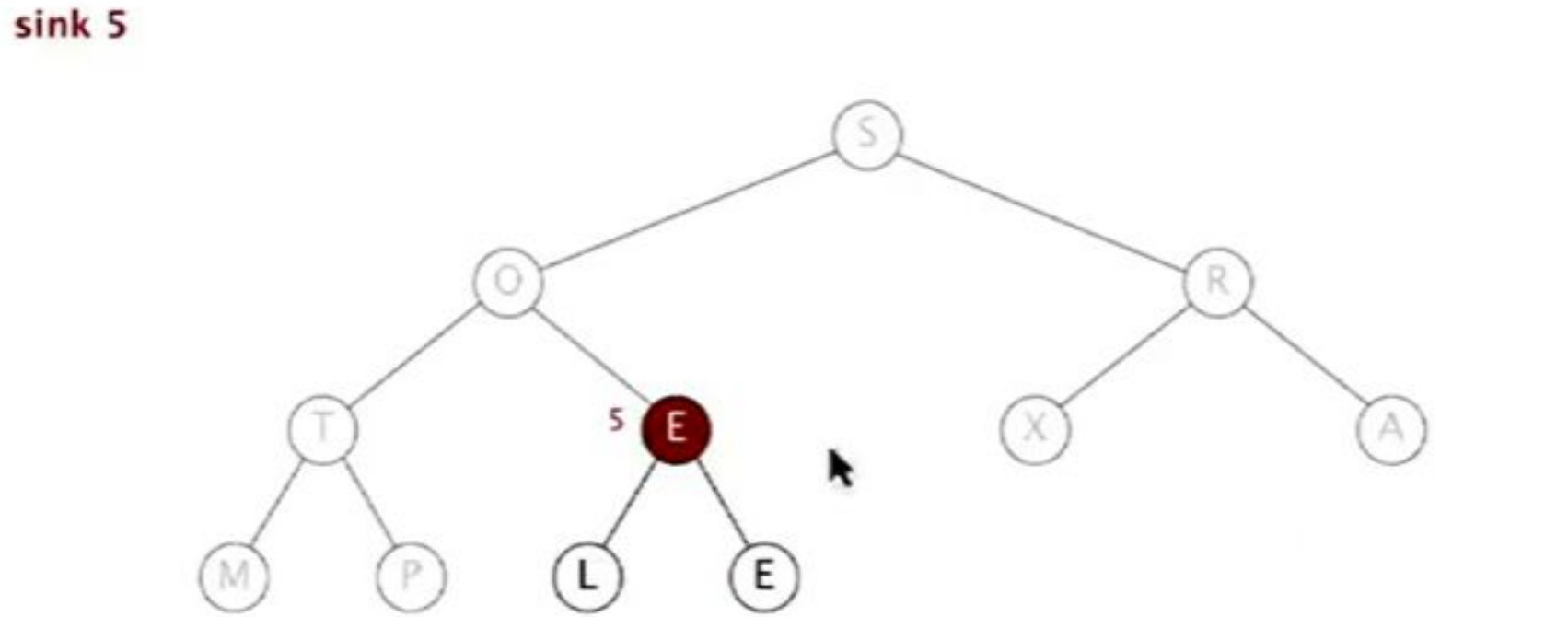
- Heap construction – build max heap using bottom-up method:



S	O	R	T	E	X	A	M	P	L	E

Heapsort demo

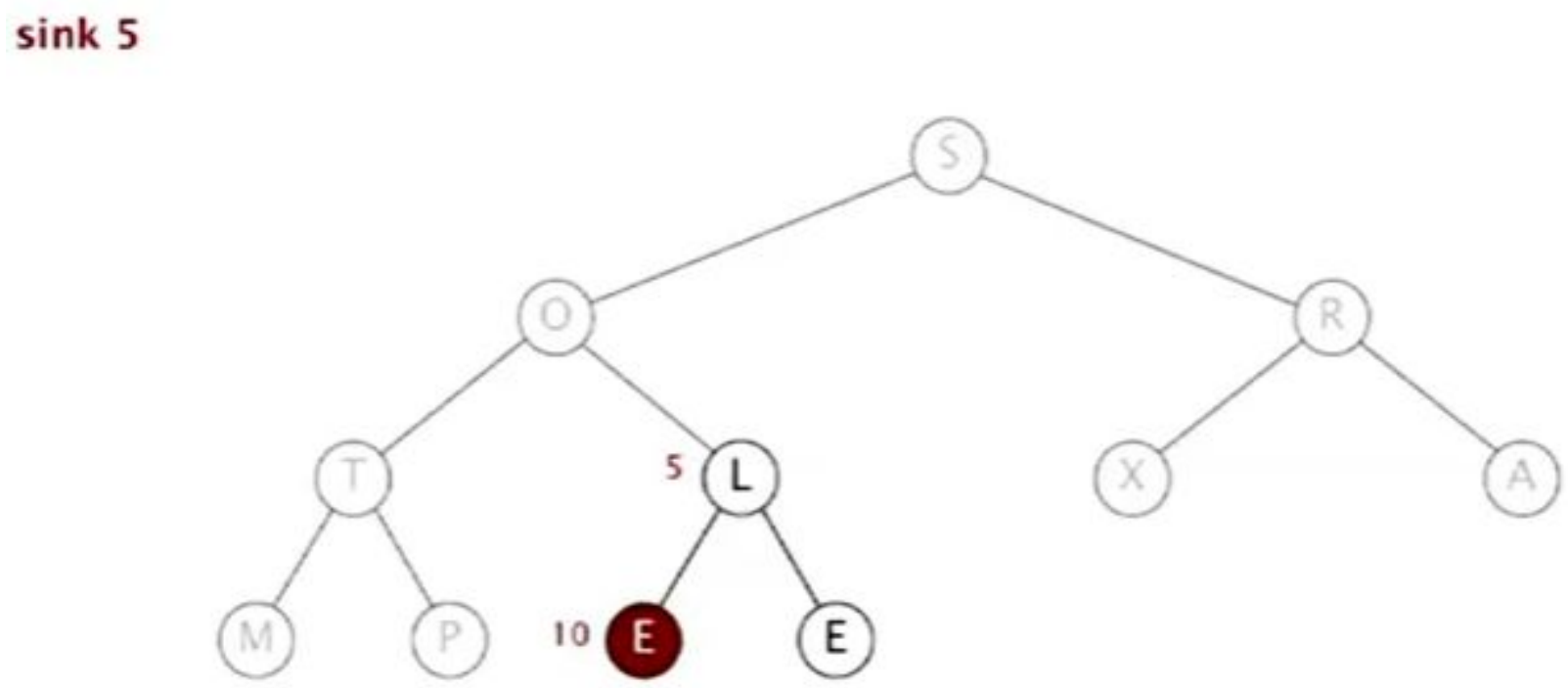
- Heap construction – build max heap using bottom-up method:



S	O	R	T	E	X	A	M	P	L	E

Heapsort demo

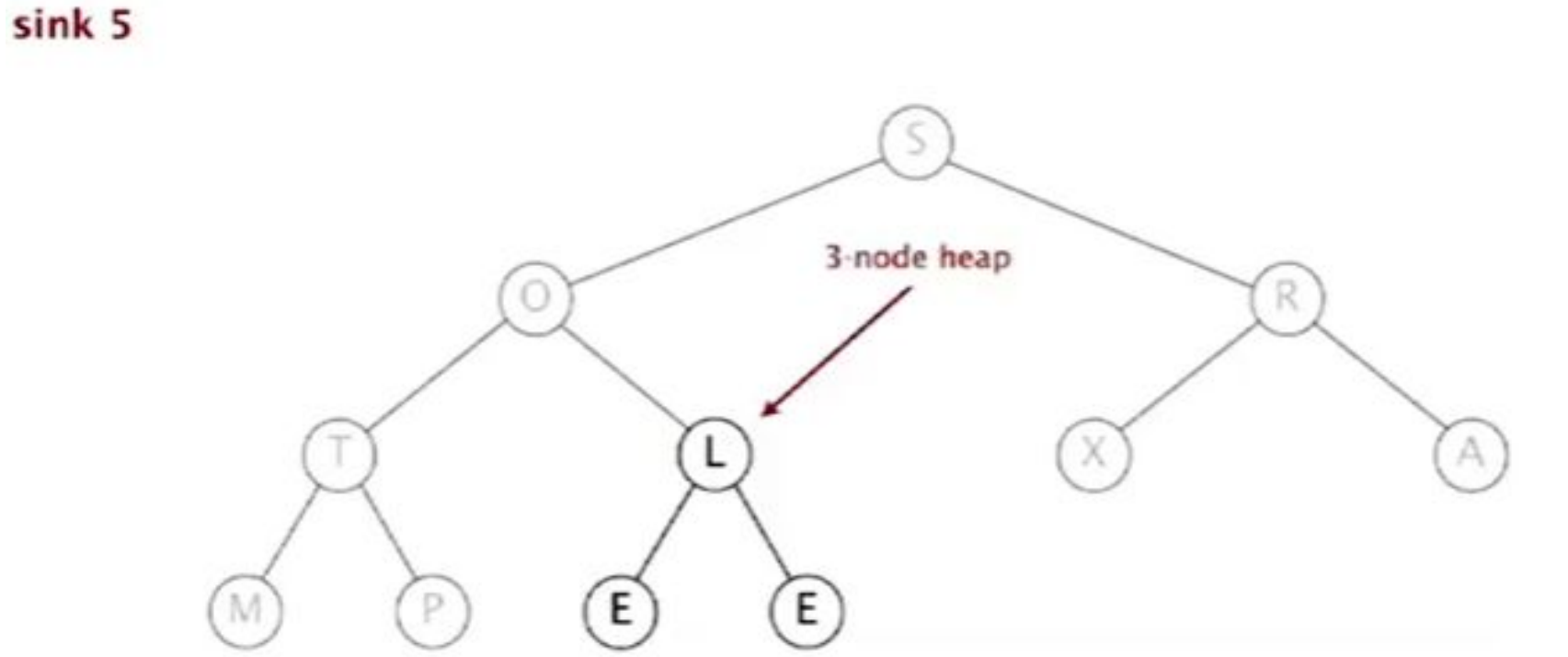
- Heap construction – build max heap using bottom-up method:



S	O	R	T	L	X	A	M	P	E	E

Heapsort demo

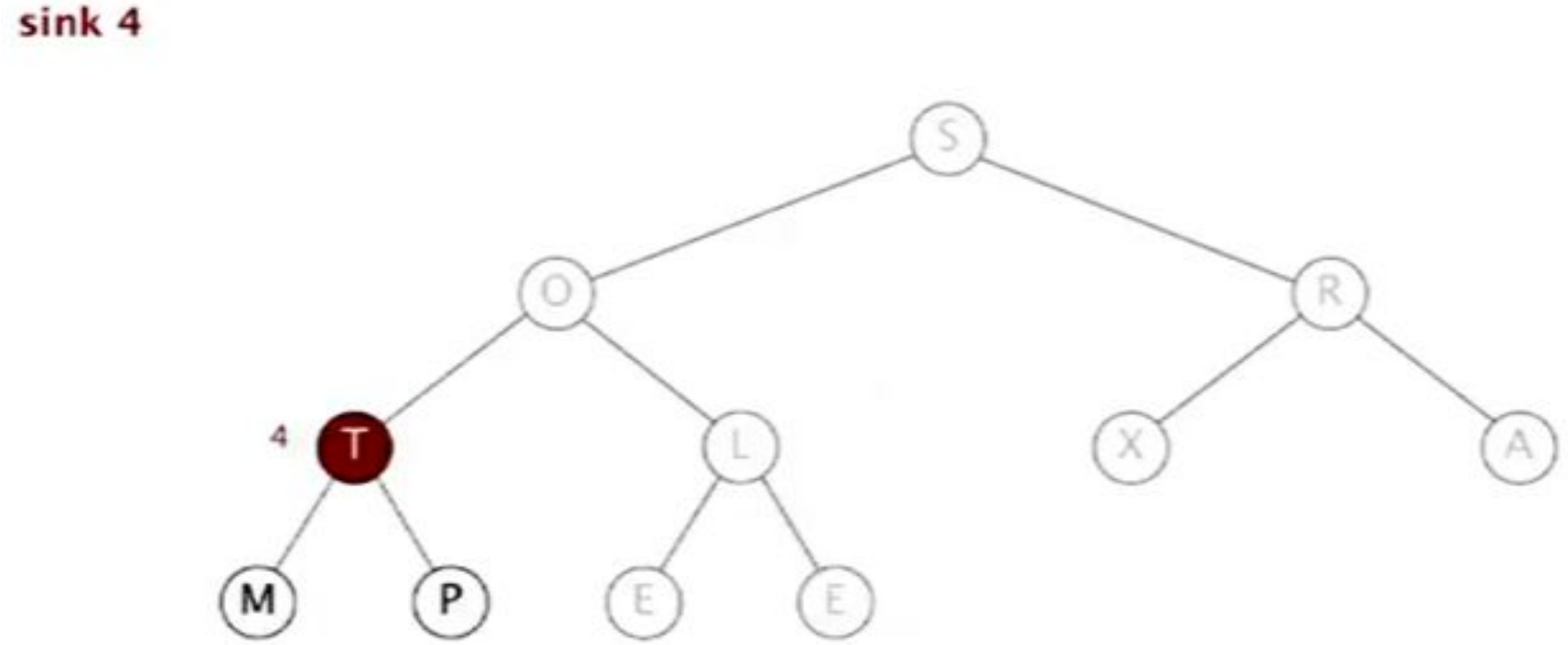
- Heap construction – build max heap using bottom-up method:



S	O	R	T	L	X	A	M	P	E	E

Heapsort demo

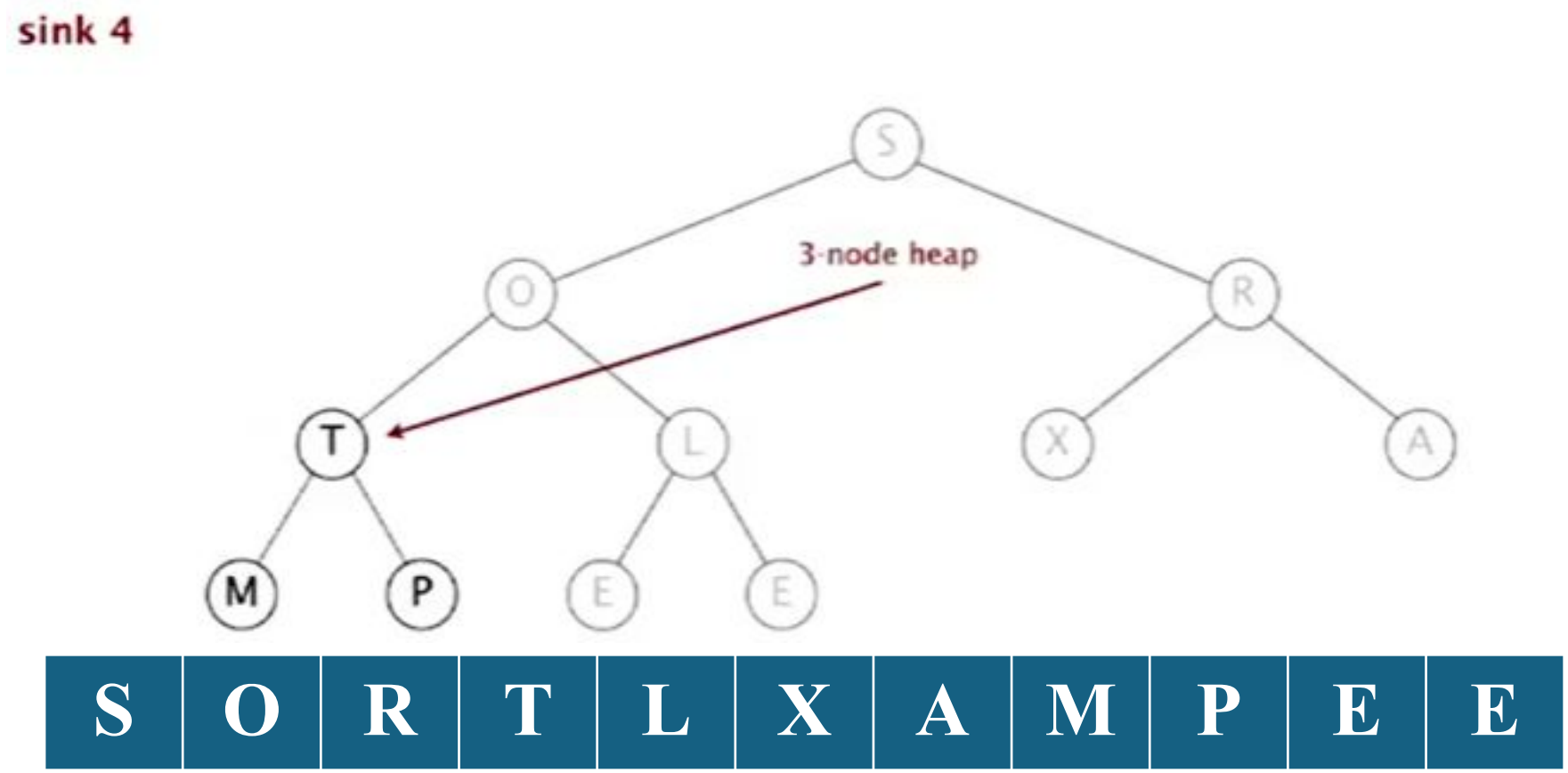
- Heap construction – build max heap using bottom-up method:



S	O	R	T	L	X	A	M	P	E	E

Heapsort demo

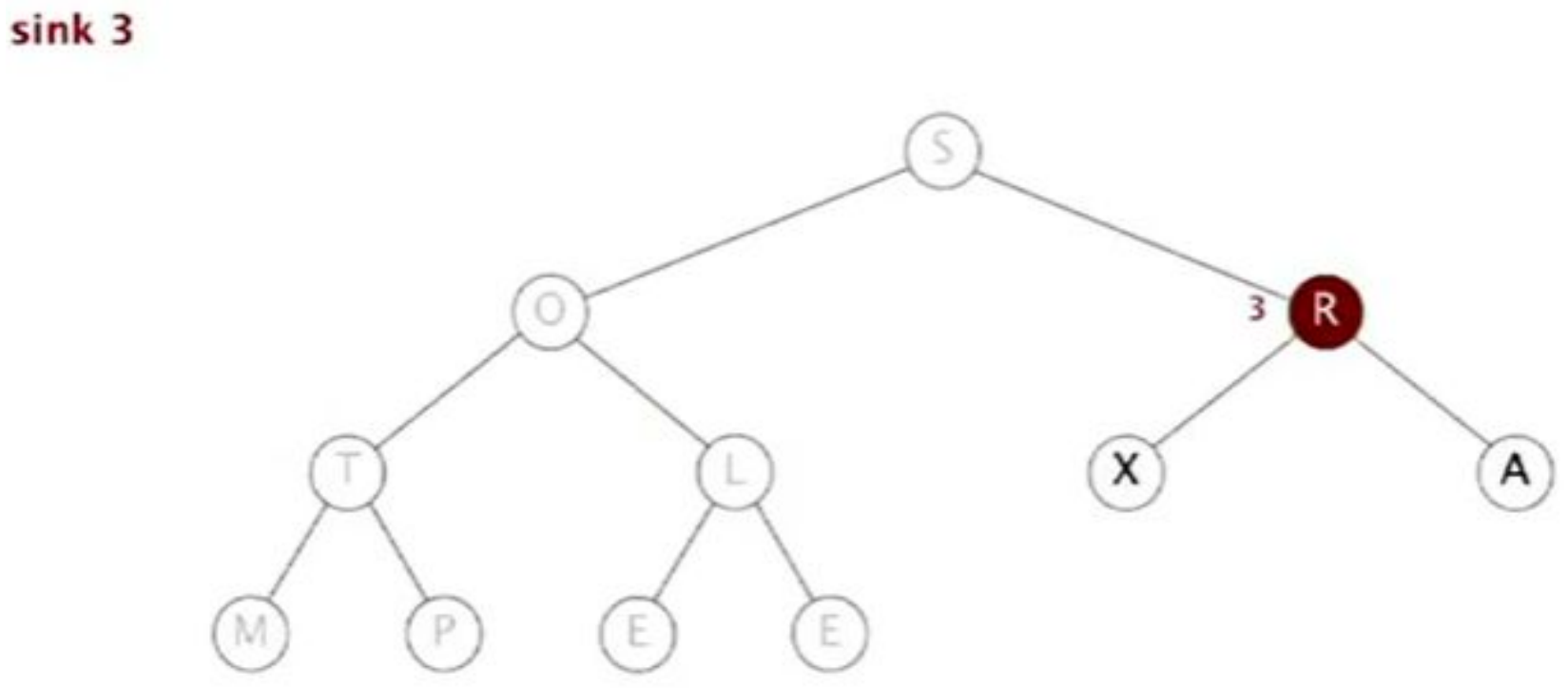
- Heap construction – build max heap using bottom-up method:



S	O	R	T	L	X	A	M	P	E	E

Heapsort demo

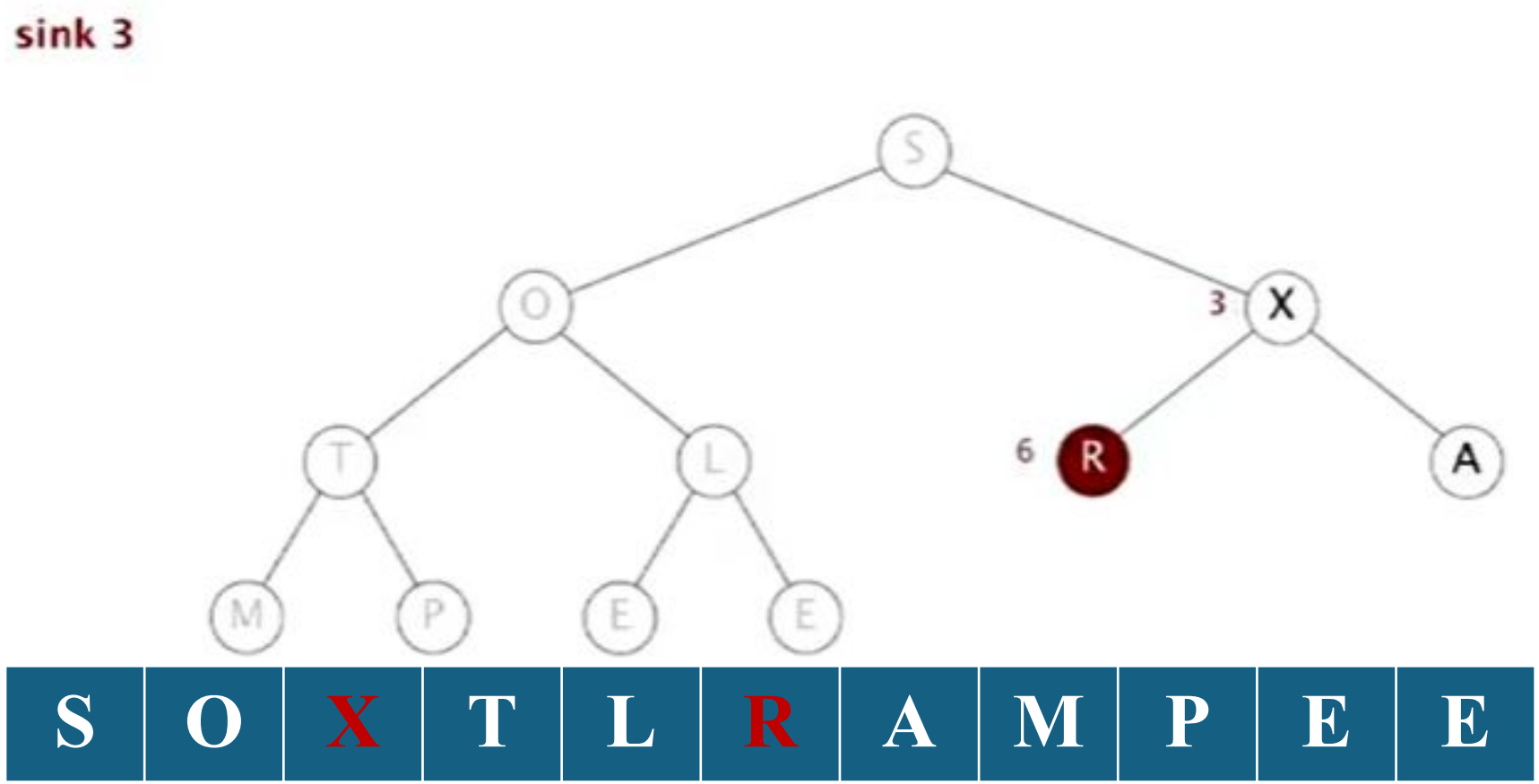
- Heap construction – build max heap using bottom-up method:



S	O	R	T	L	X	A	M	P	E	E

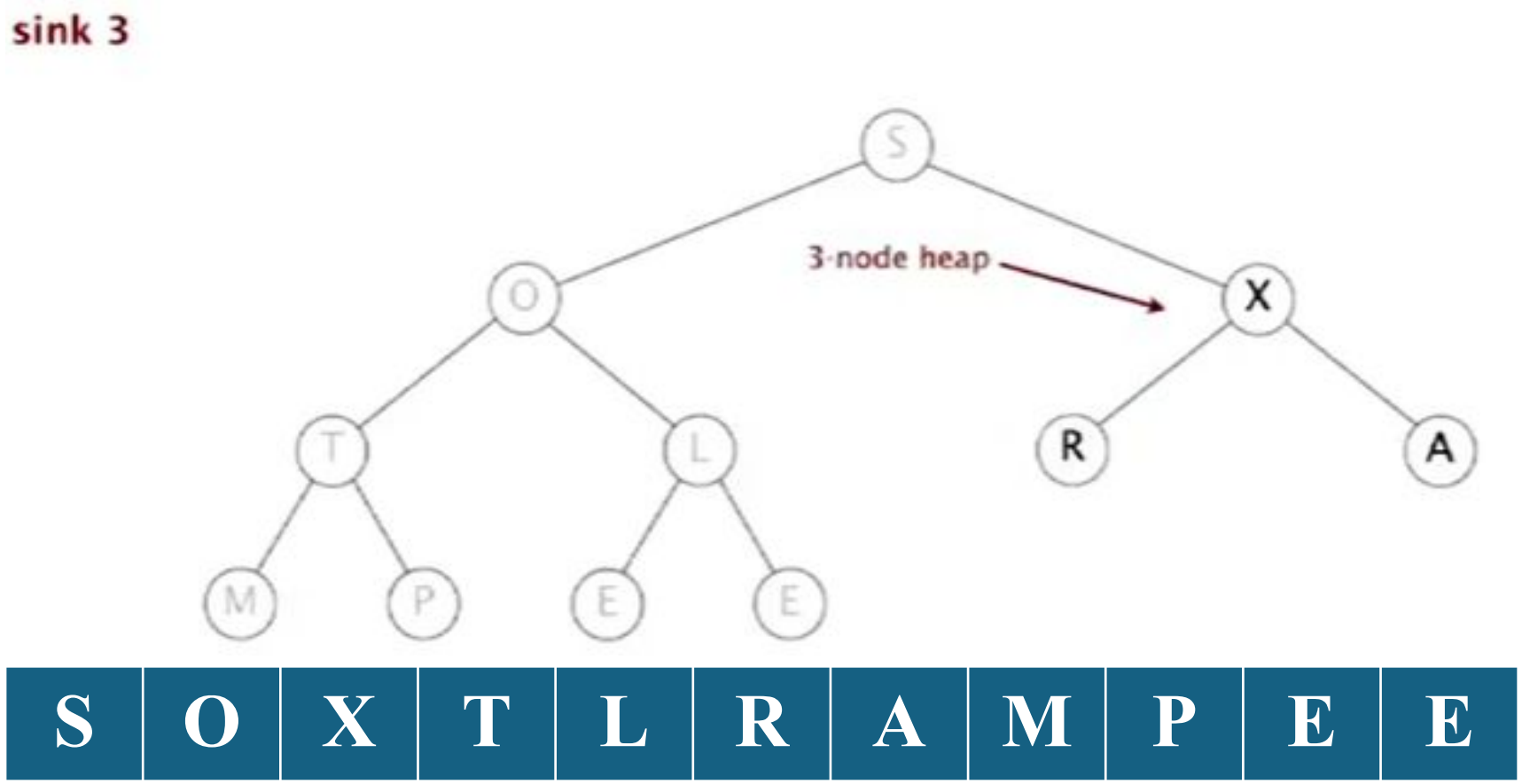
Heapsort demo

- Heap construction – build max heap using bottom-up method:



Heapsort demo

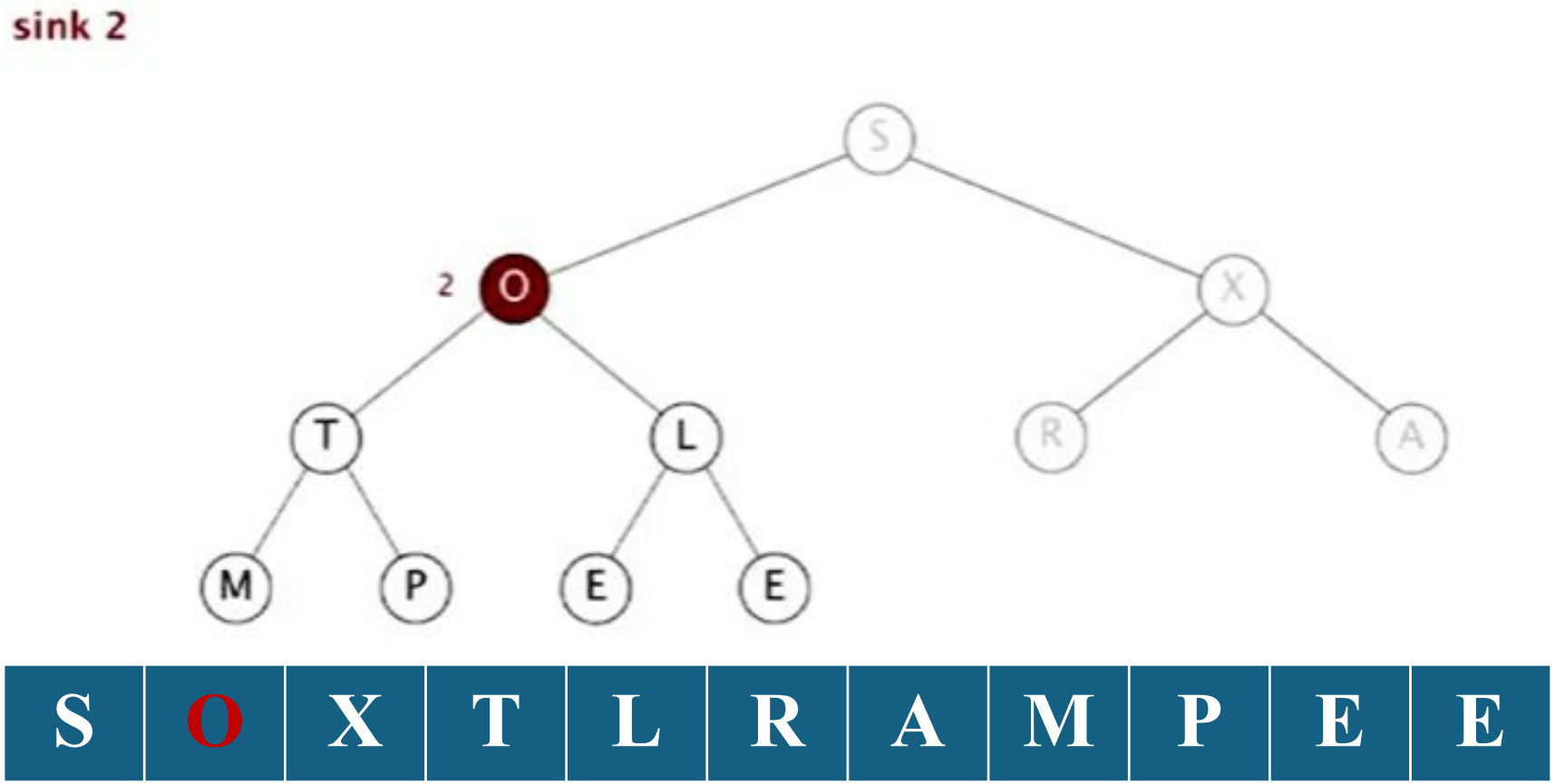
- Heap construction – build max heap using bottom-up method:



S	O	X	T	L	R	A	M	P	E	E

Heapsort demo

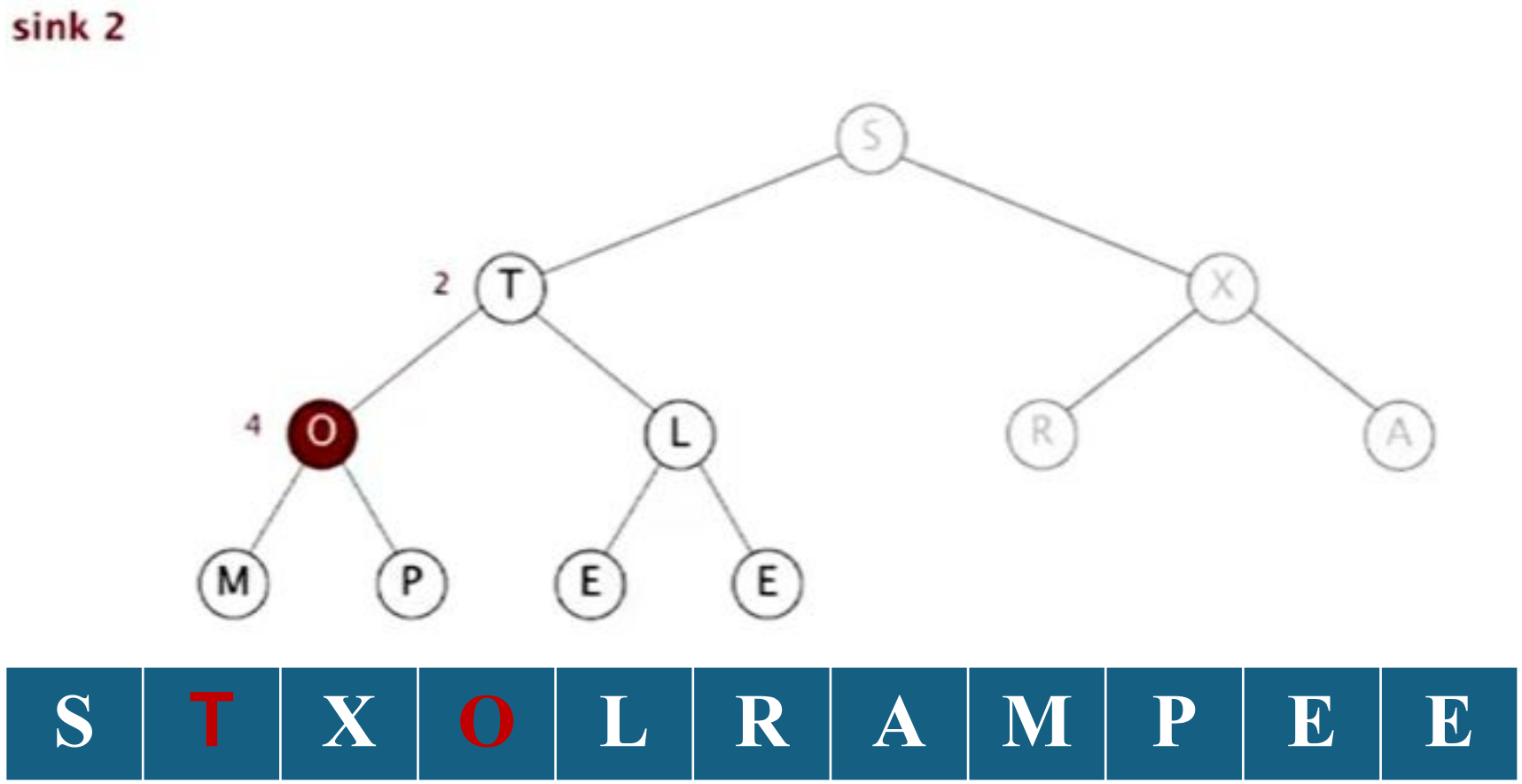
- Heap construction – build max heap using bottom-up method:



S	O	X	T	L	R	A	M	P	E	E

Heapsort demo

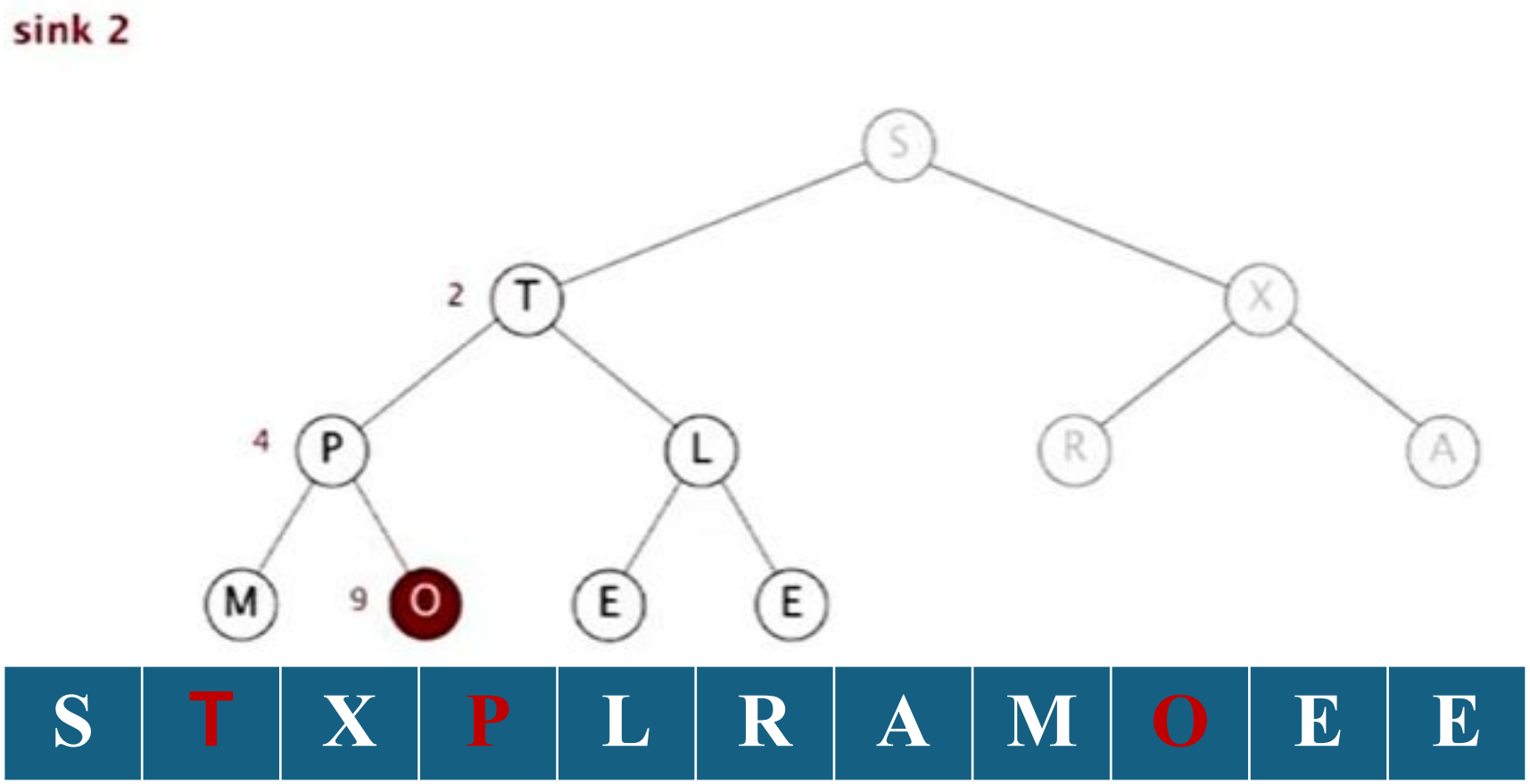
- Heap construction – build max heap using bottom-up method:



S	T	X	O	L	R	A	M	P	E	E

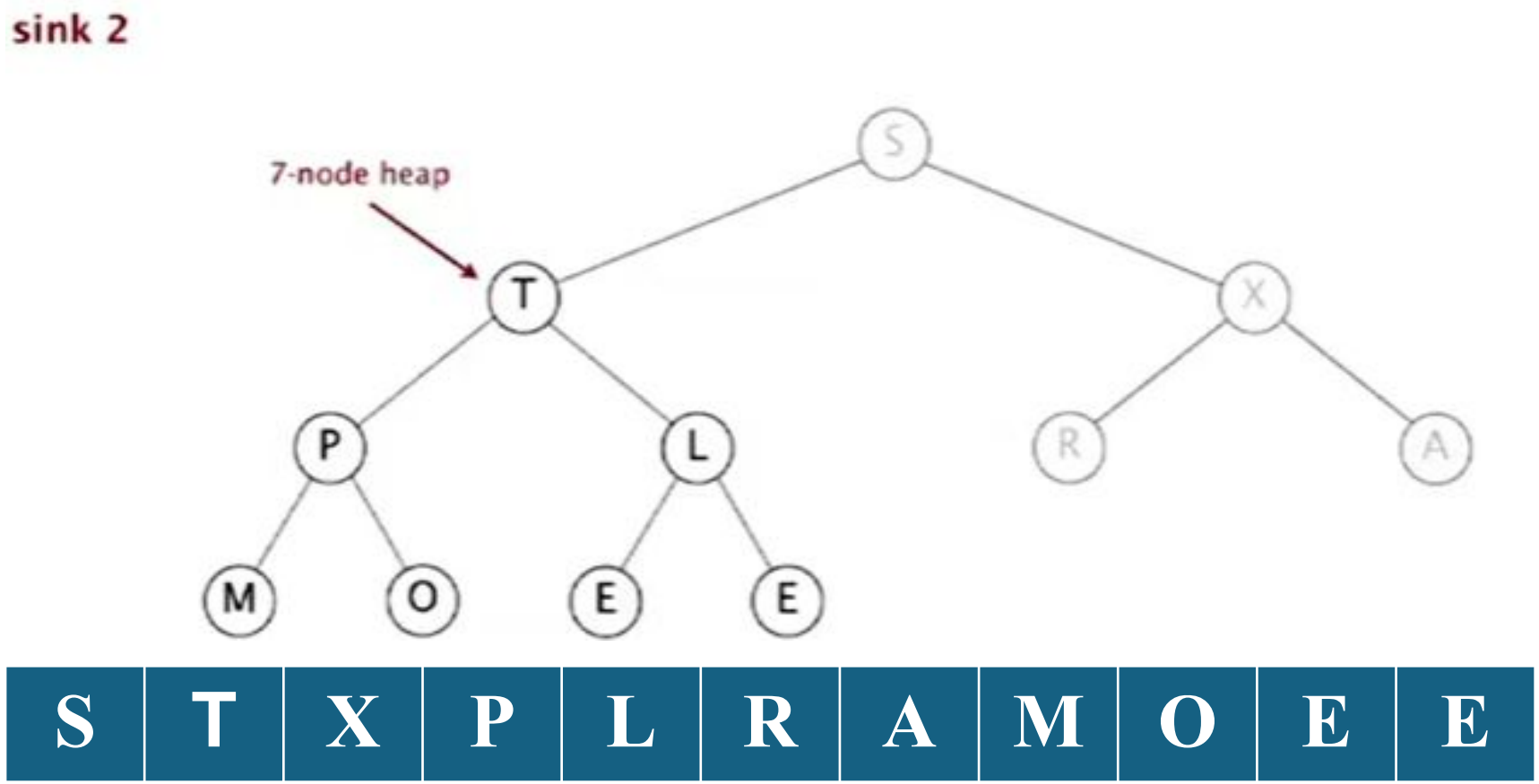
Heapsort demo

- Heap construction – build max heap using bottom-up method:



Heapsort demo

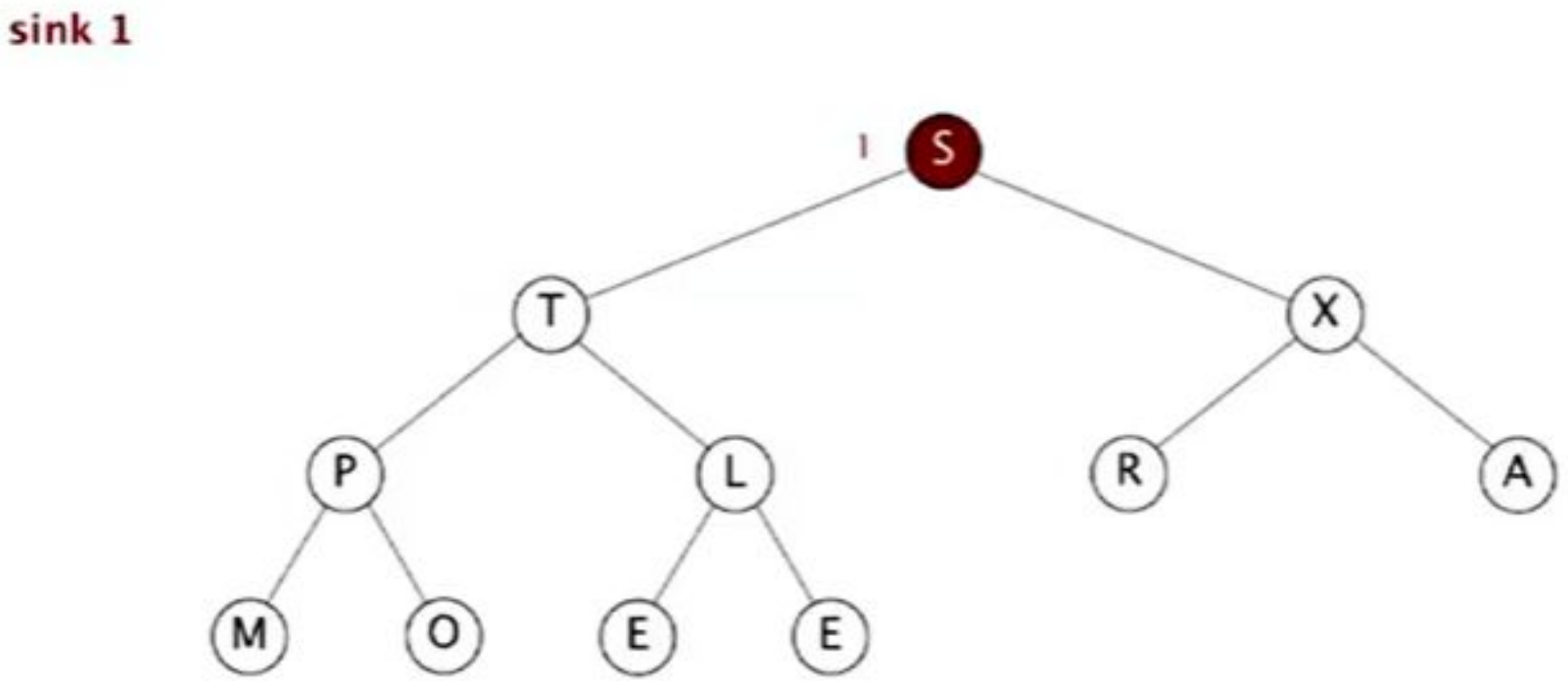
- Heap construction – build max heap using bottom-up method:



S	T	X	P	L	R	A	M	O	E	E

Heapsort demo

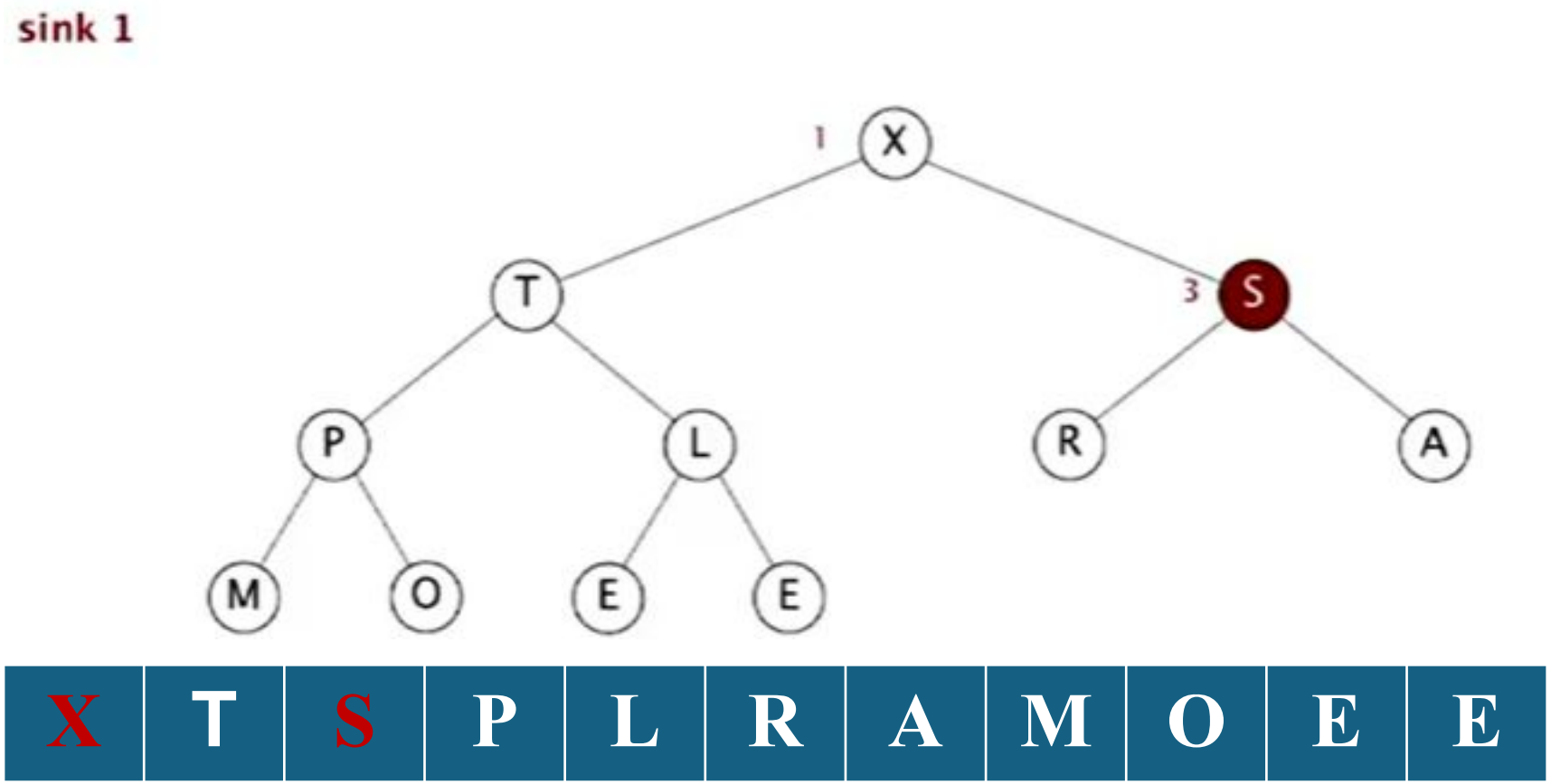
- Heap construction – build max heap using bottom-up method:



S	T	X	P	L	R	A	M	O	E	E

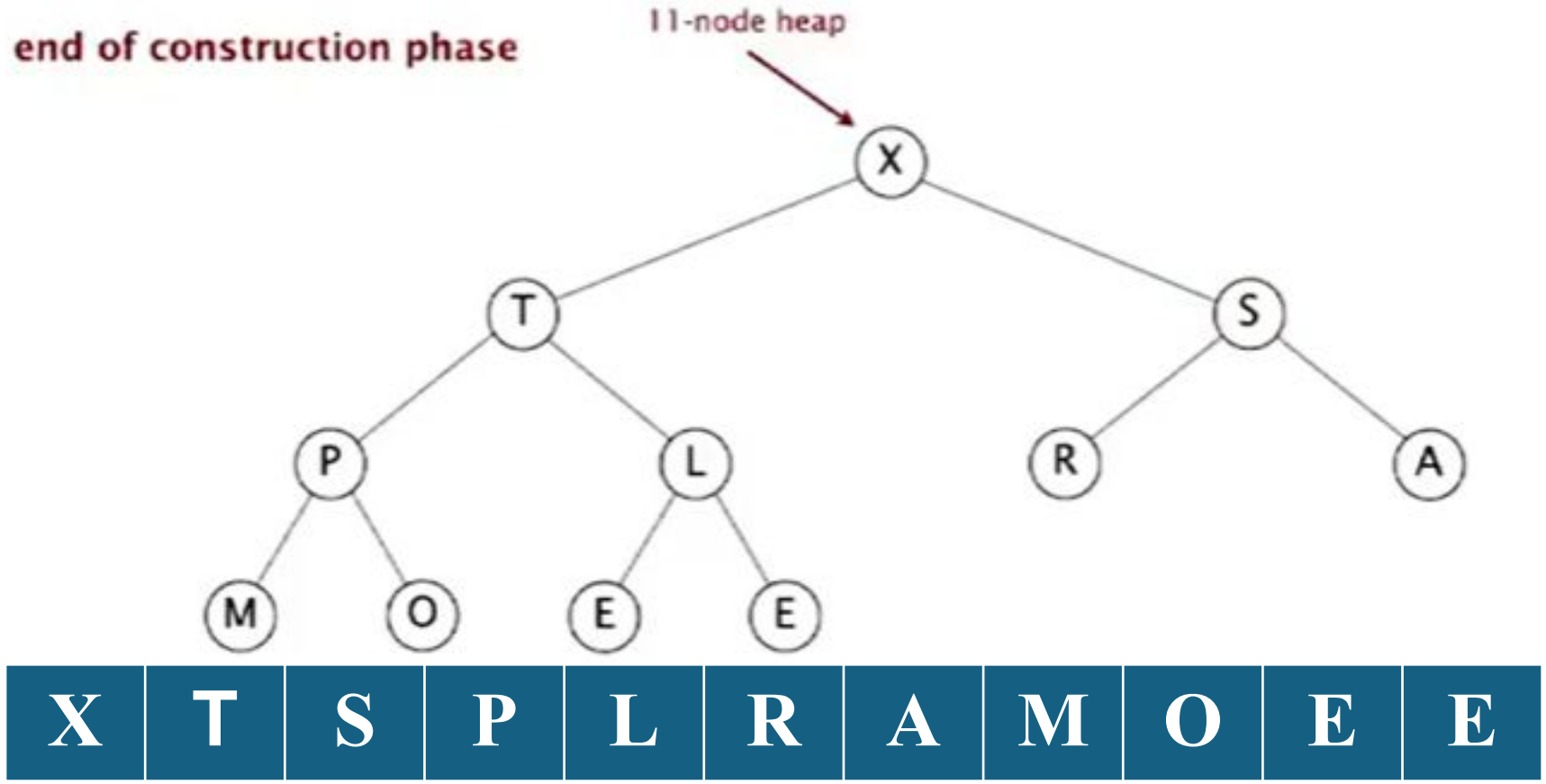
Heapsort demo

- Heap construction – build max heap using bottom-up method:



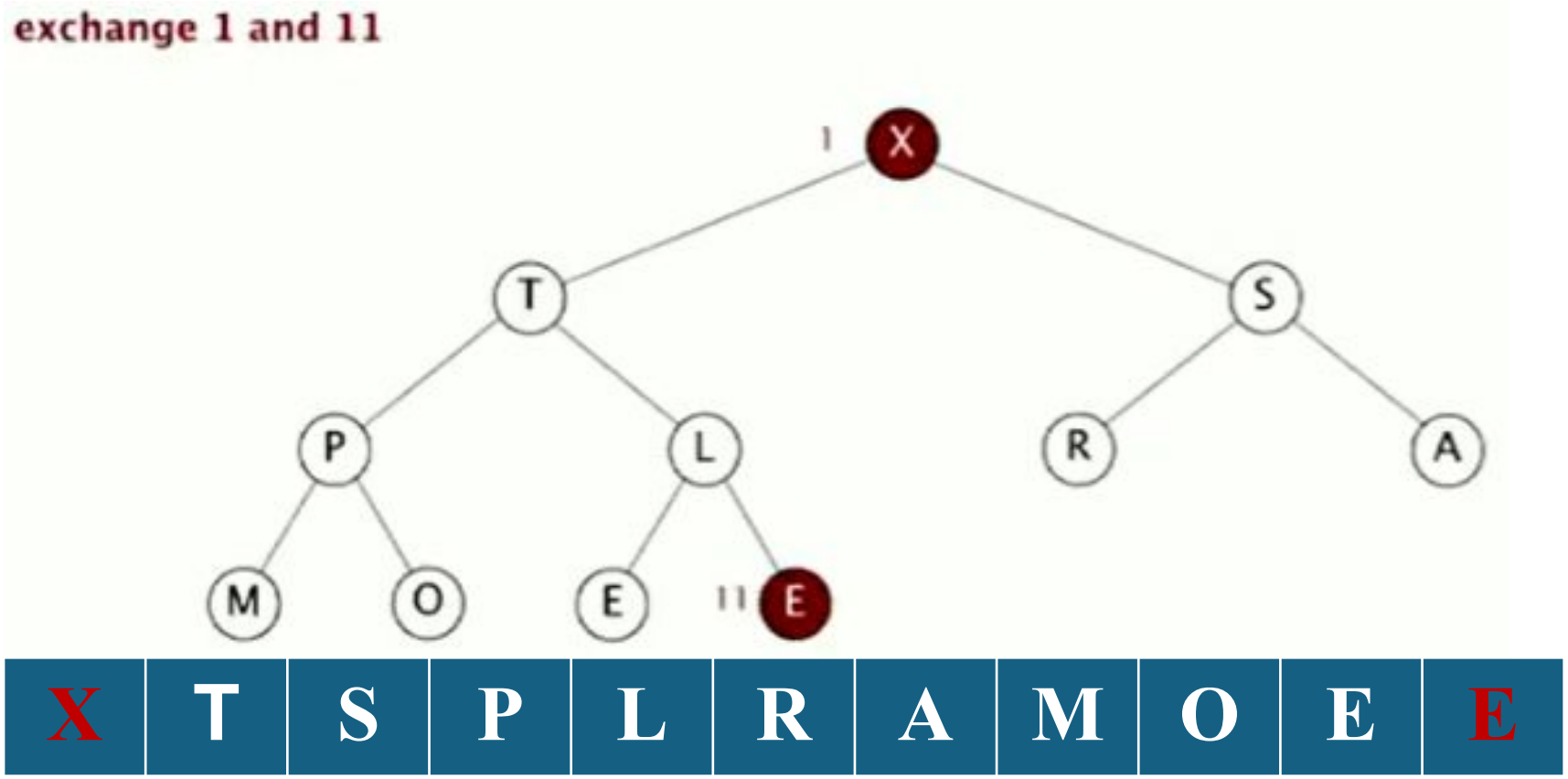
Heapsort demo

- Heap construction – build max heap using bottom-up method:



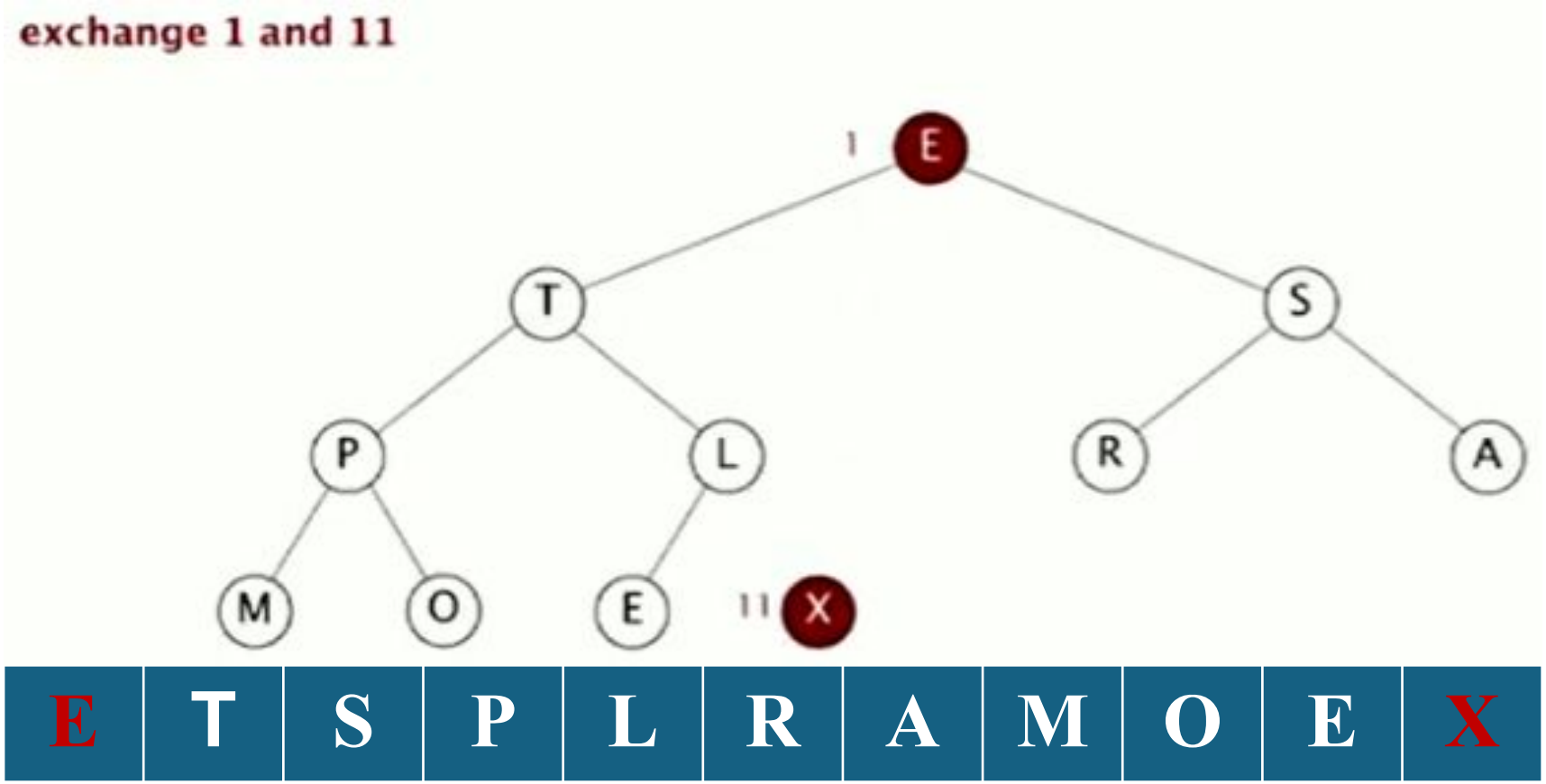
Heapsort demo

- Sortdown: Repeatedly delete the largest remaining item



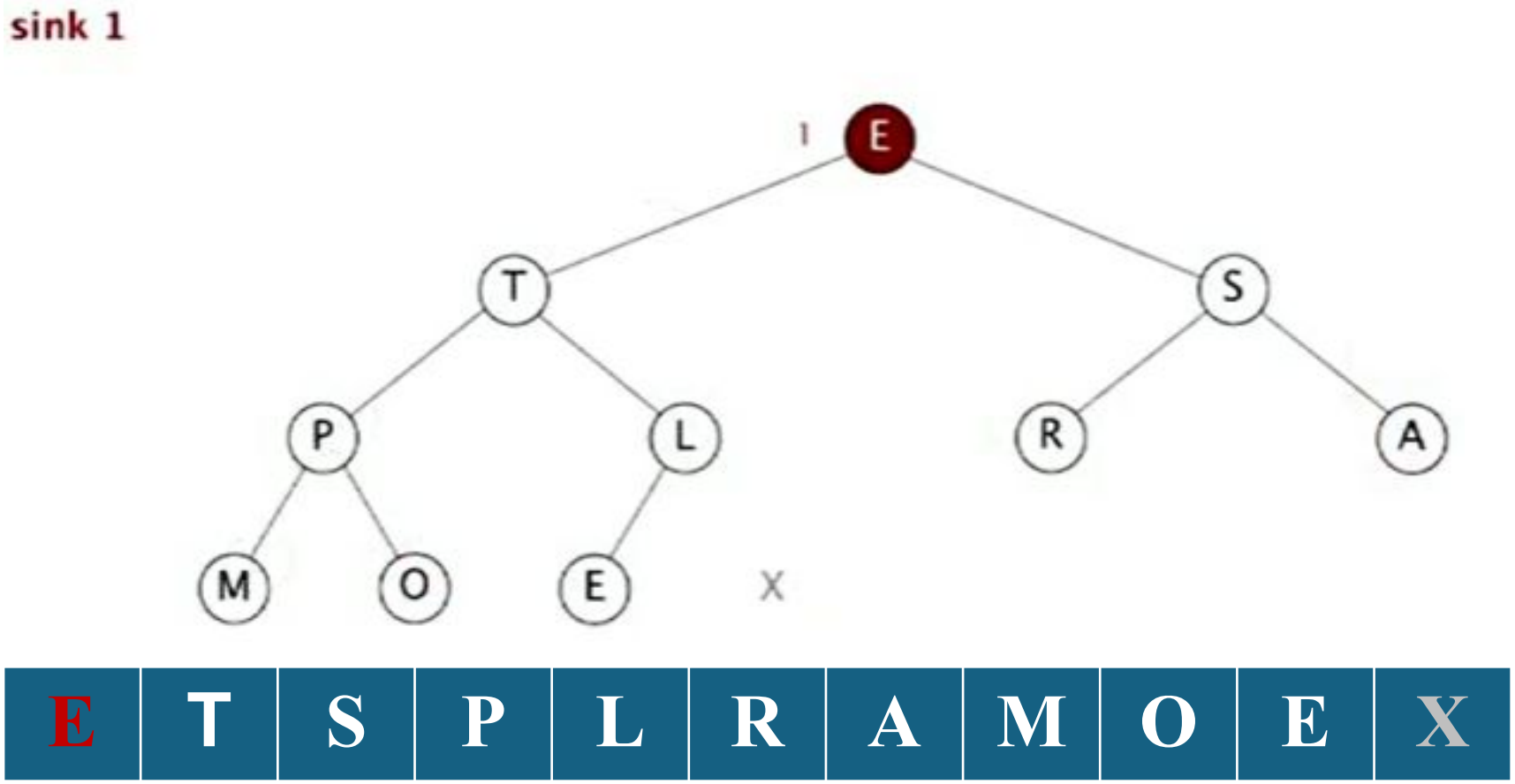
Heapsort demo

- Sortdown: Repeatedly delete the largest remaining item



Heapsort demo

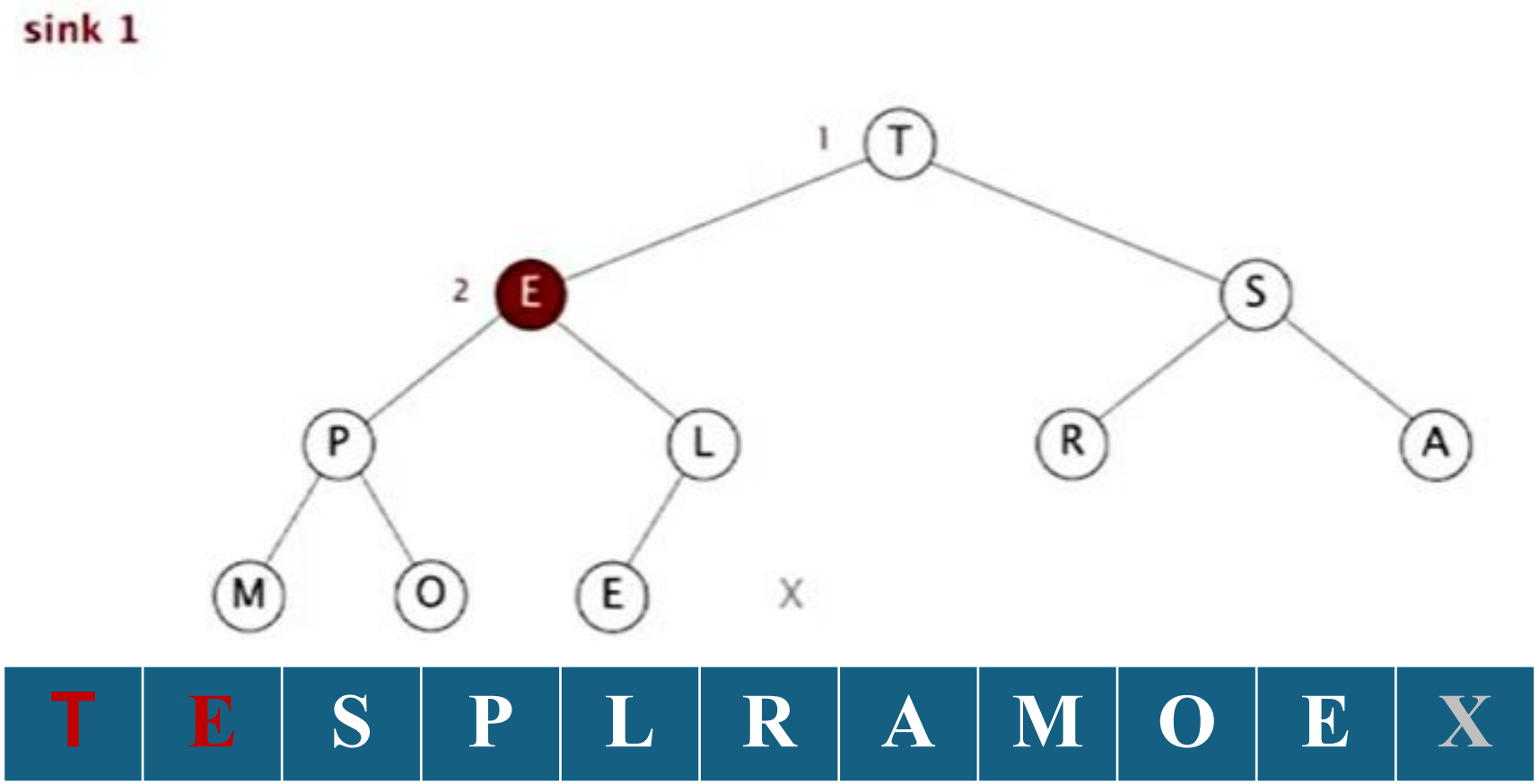
- Sortdown: Repeatedly delete the largest remaining item



E	T	S	P	L	R	A	M	O	E	X

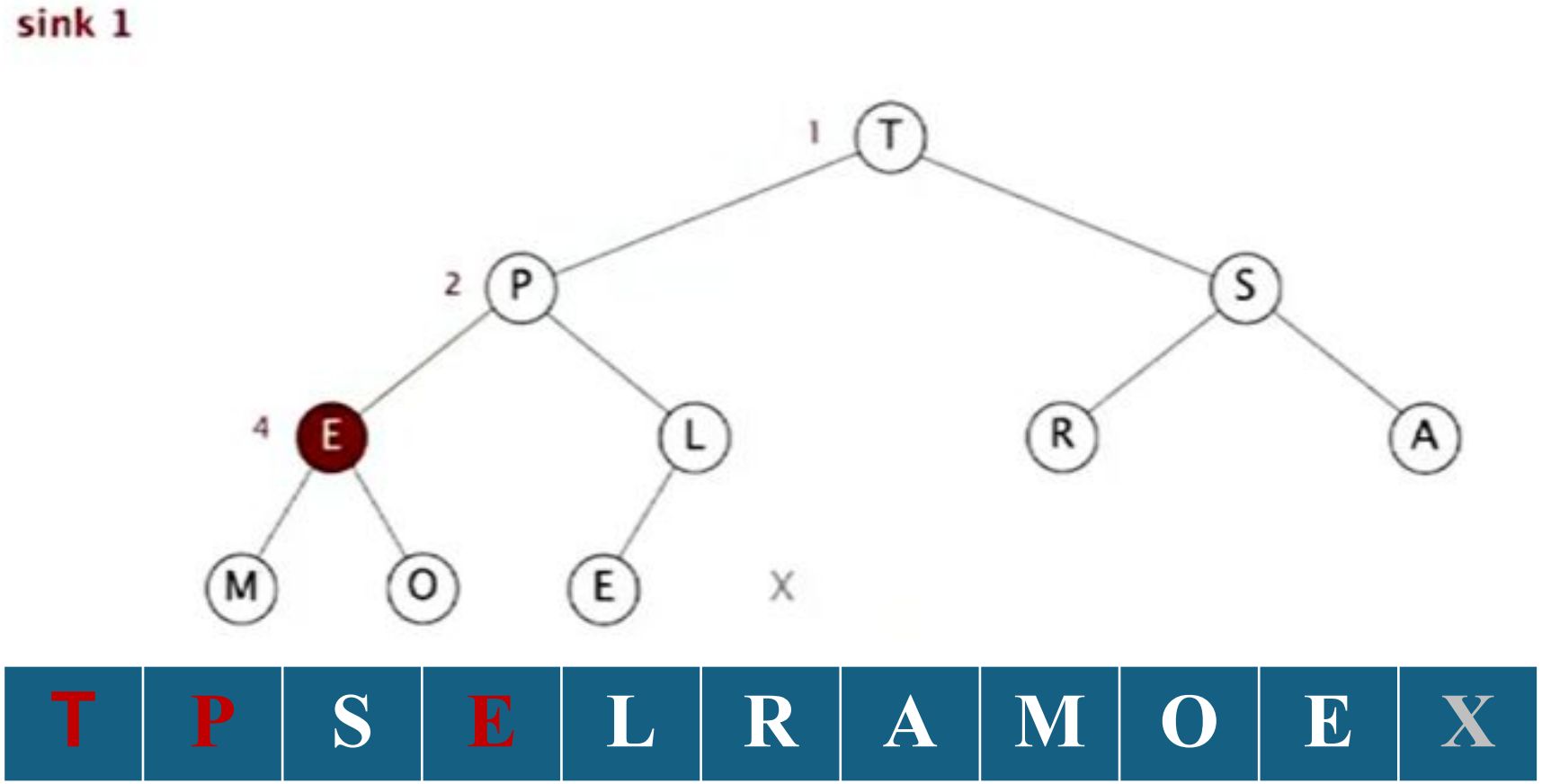
Heapsort demo

- Sortdown: Repeatedly delete the largest remaining item



Heapsort demo

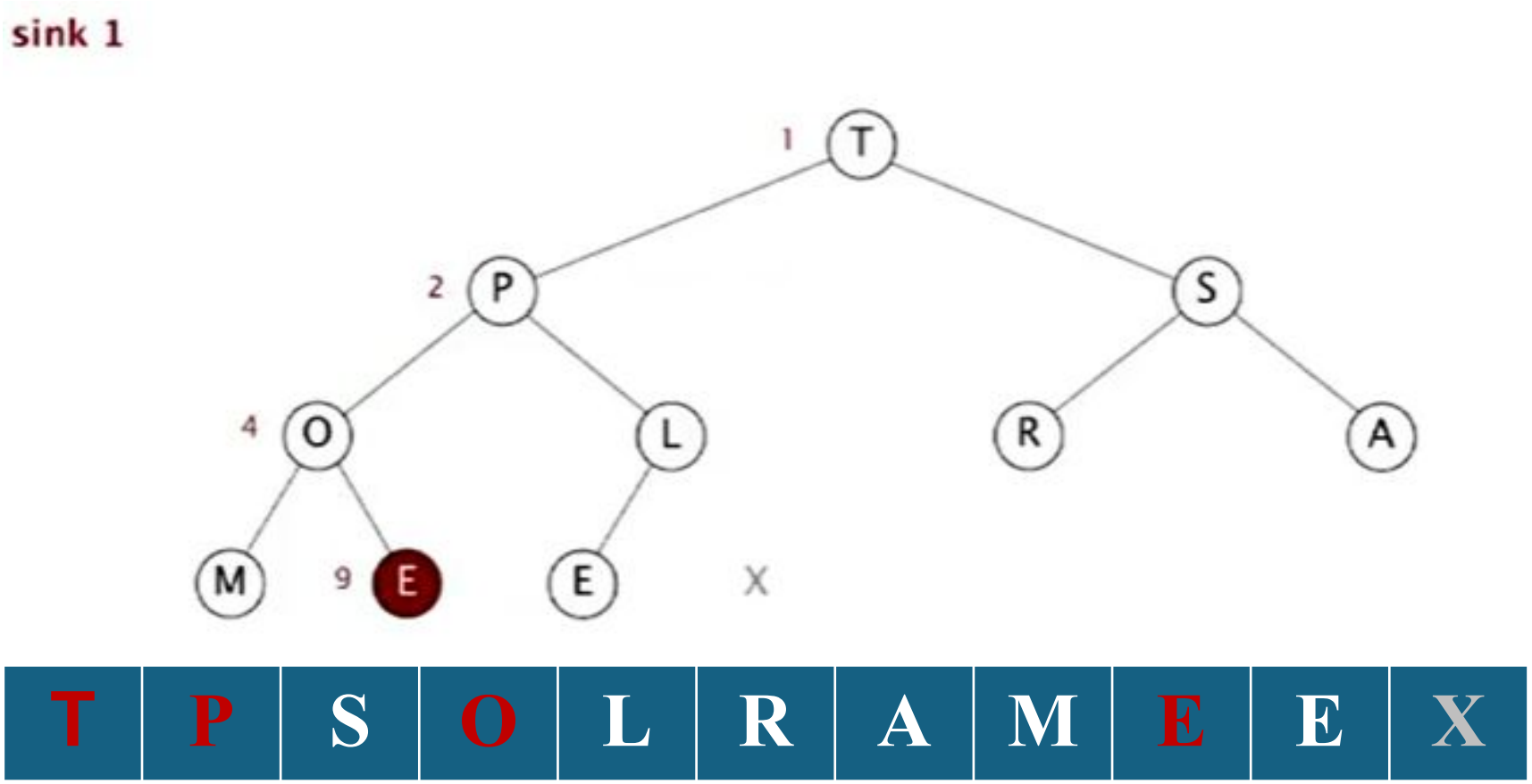
- Sortdown: Repeatedly delete the largest remaining item



T	P	S	E	L	R	A	M	O	E	X

Heapsort demo

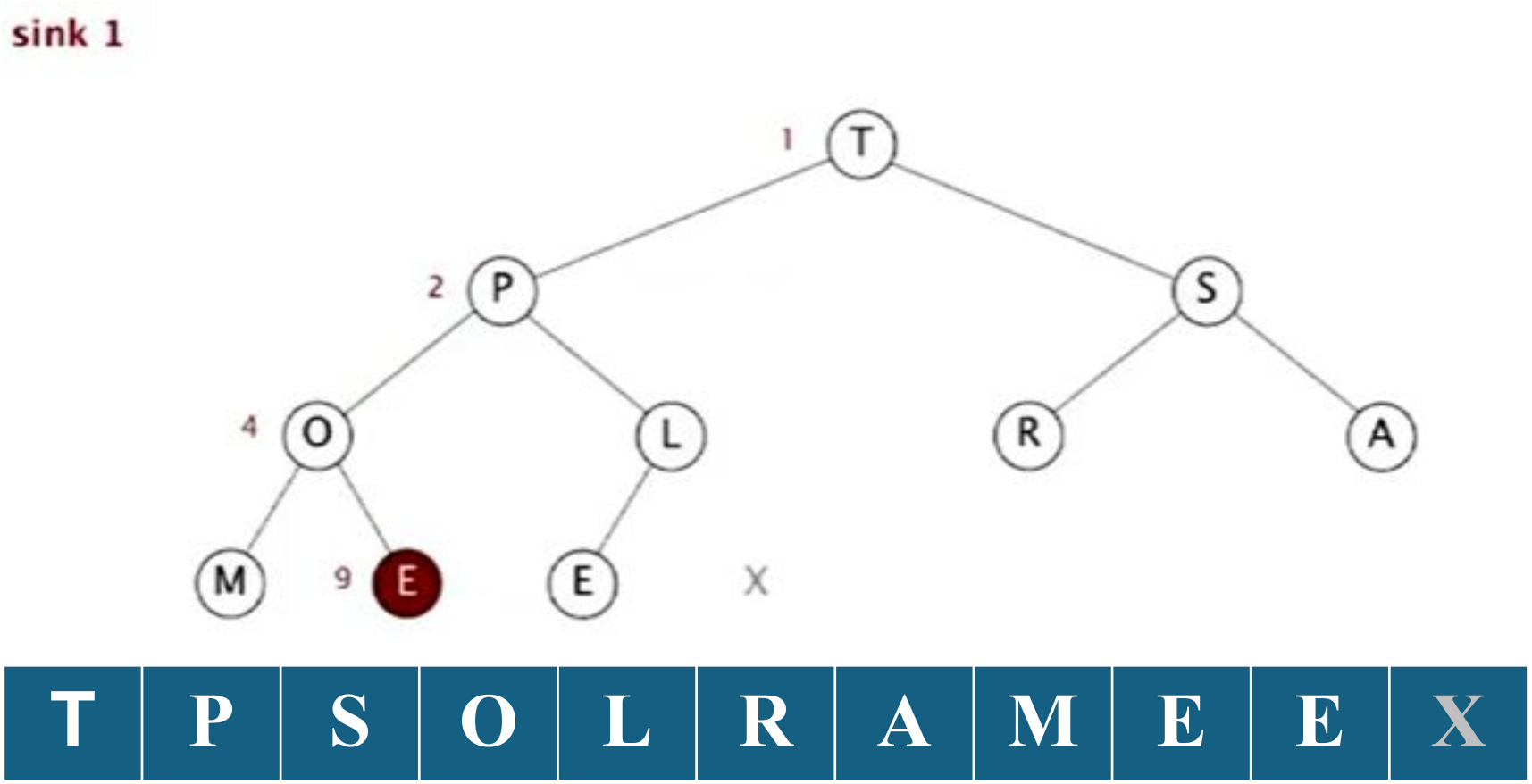
- Sortdown: Repeatedly delete the largest remaining item



T	P	S	O	L	R	A	M	E	E	X

Heapsort demo

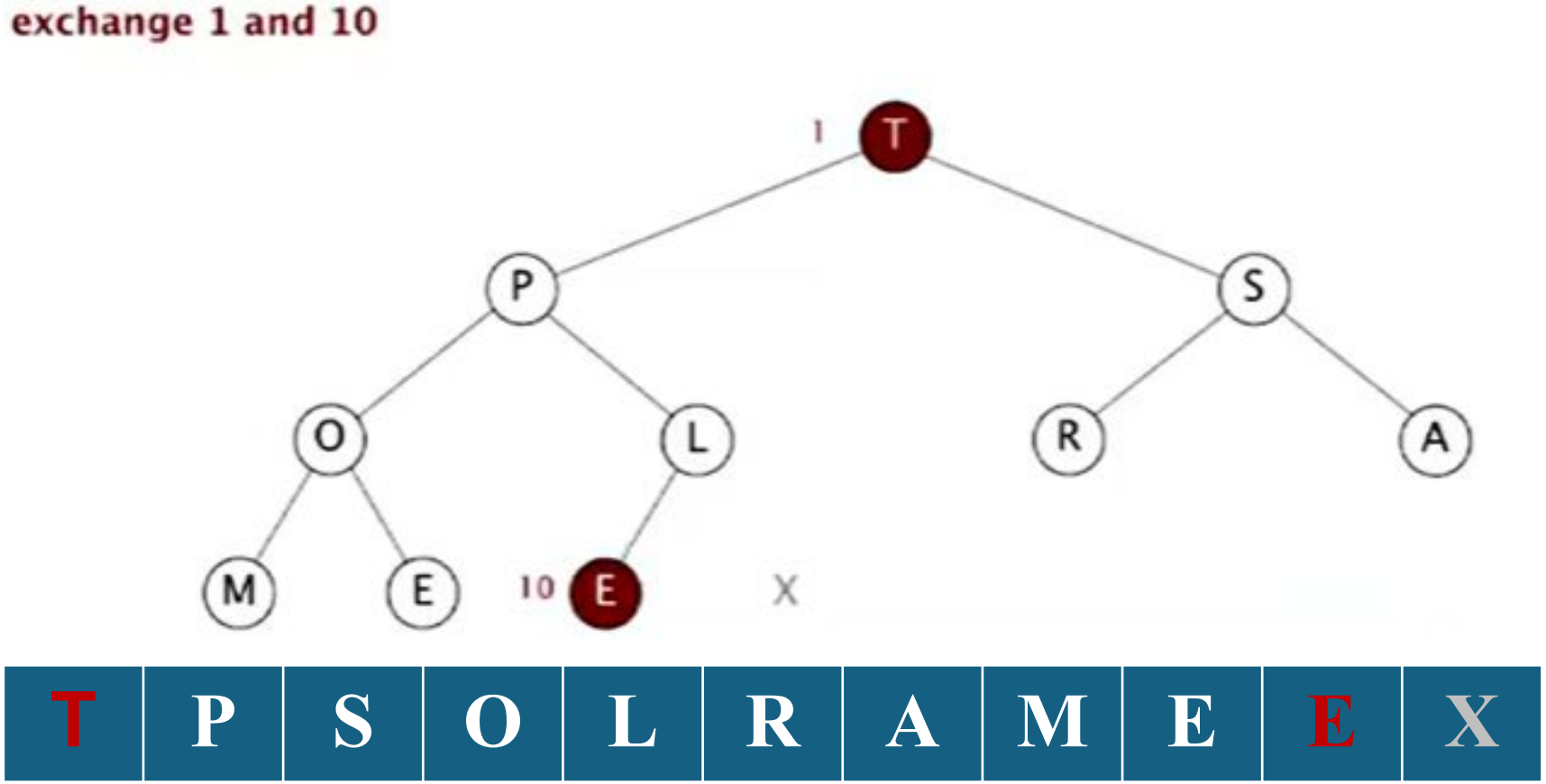
- Sortdown: Repeatedly delete the largest remaining item



T	P	S	O	L	R	A	M	E	E	X

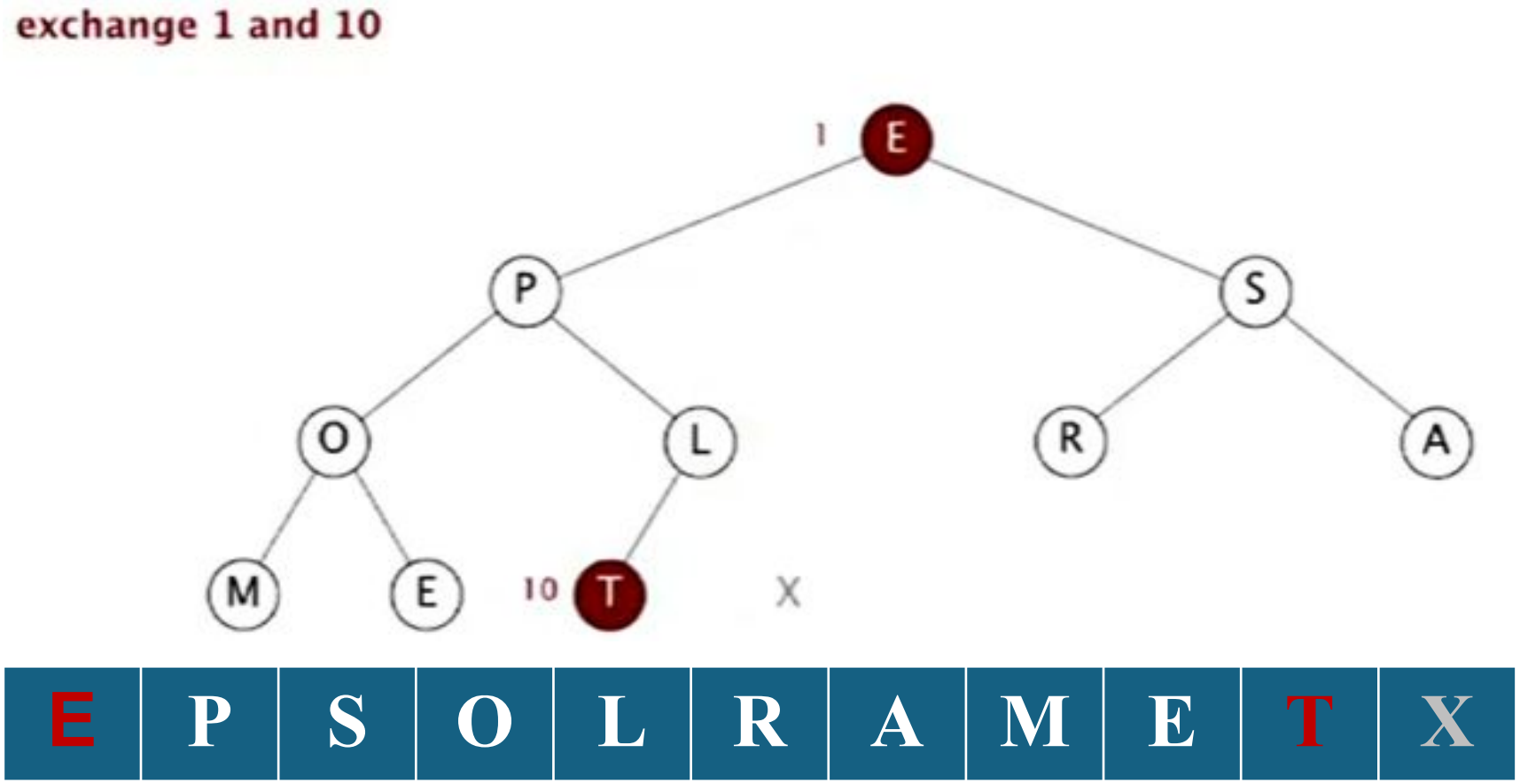
Heapsort demo

- Sortdown: Repeatedly delete the largest remaining item



Heapsort demo

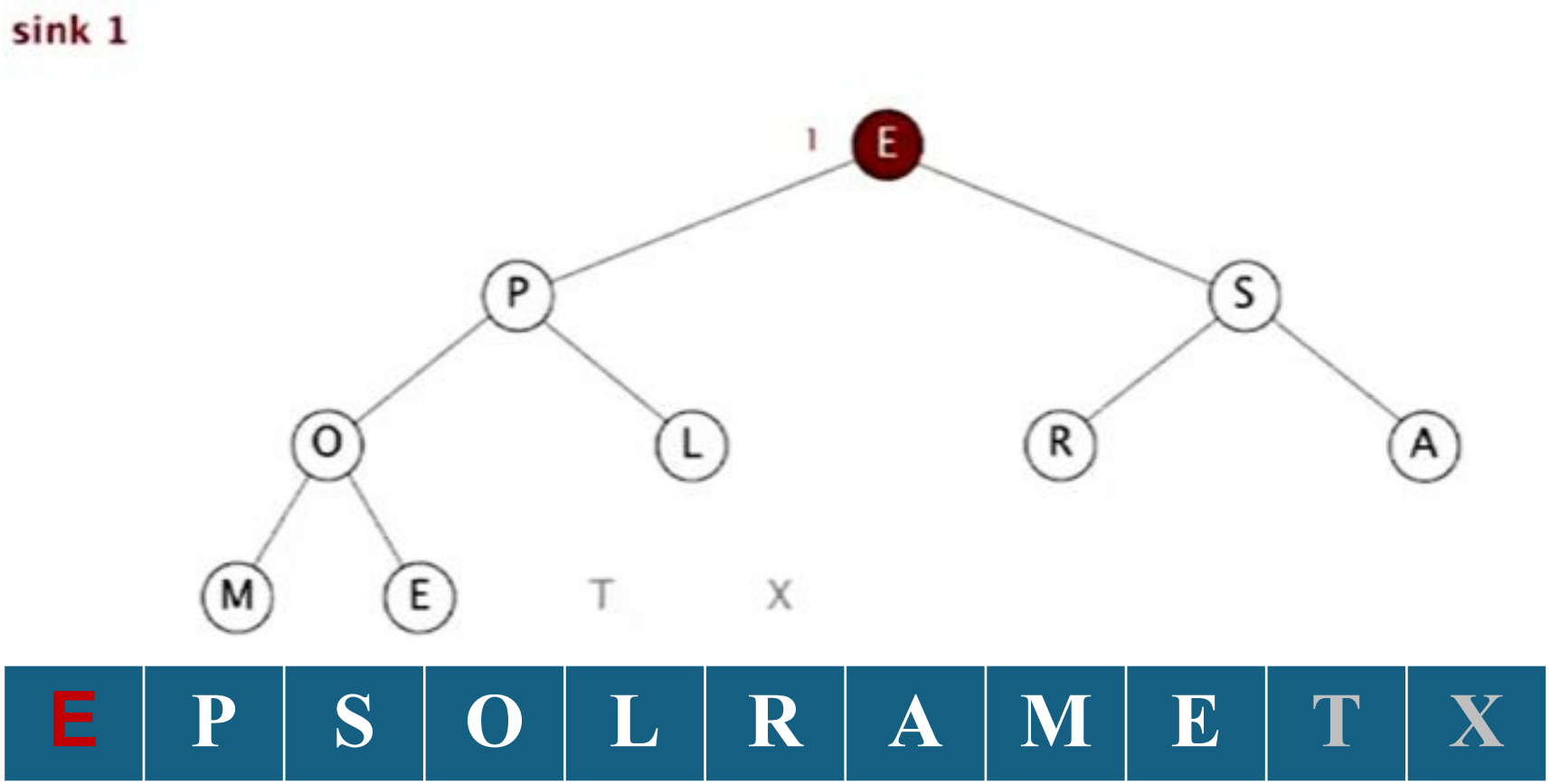
- Sortdown: Repeatedly delete the largest remaining item



E	P	S	O	L	R	A	M	E	T	X

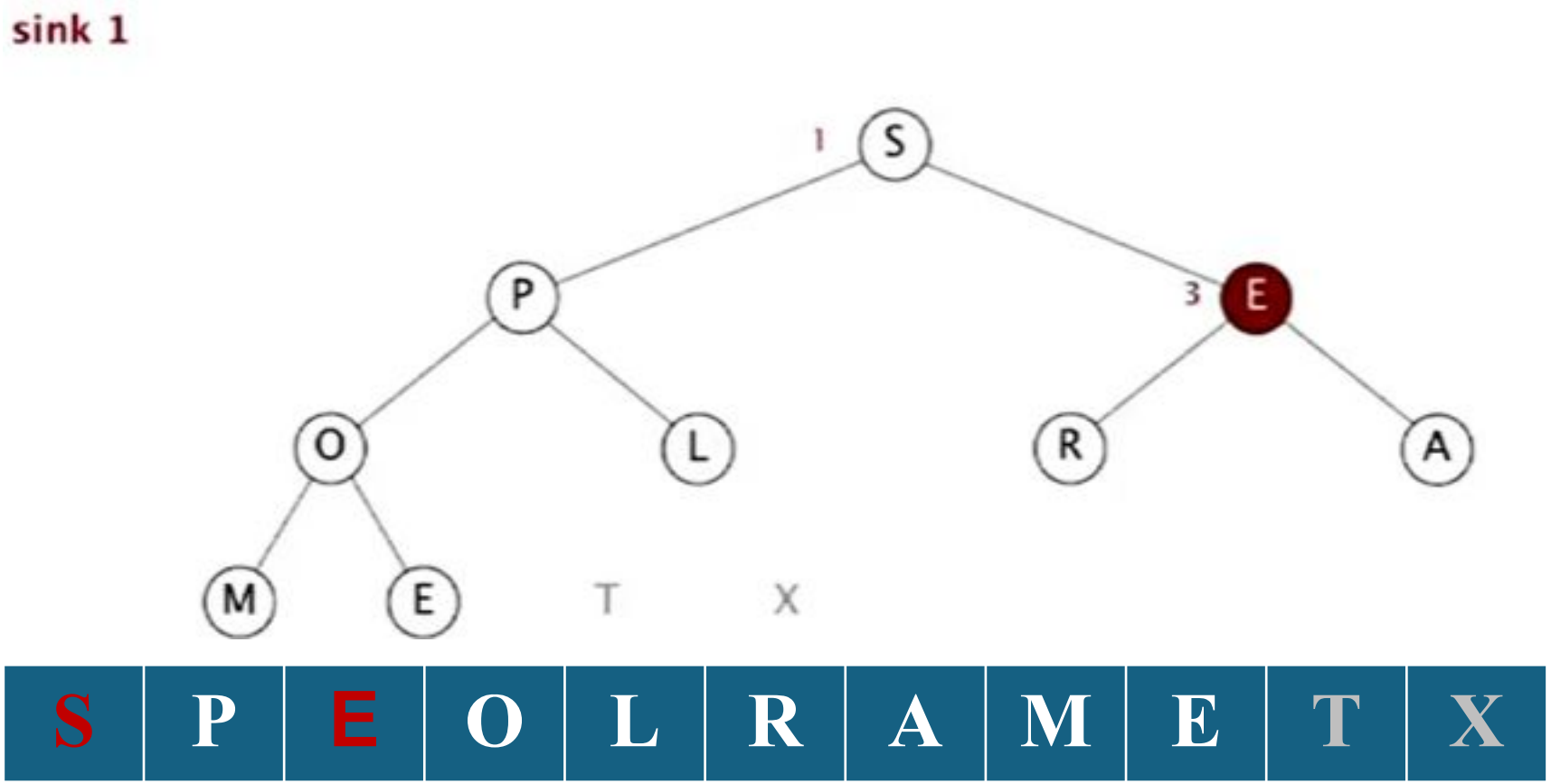
Heapsort demo

- Sortdown: Repeatedly delete the largest remaining item



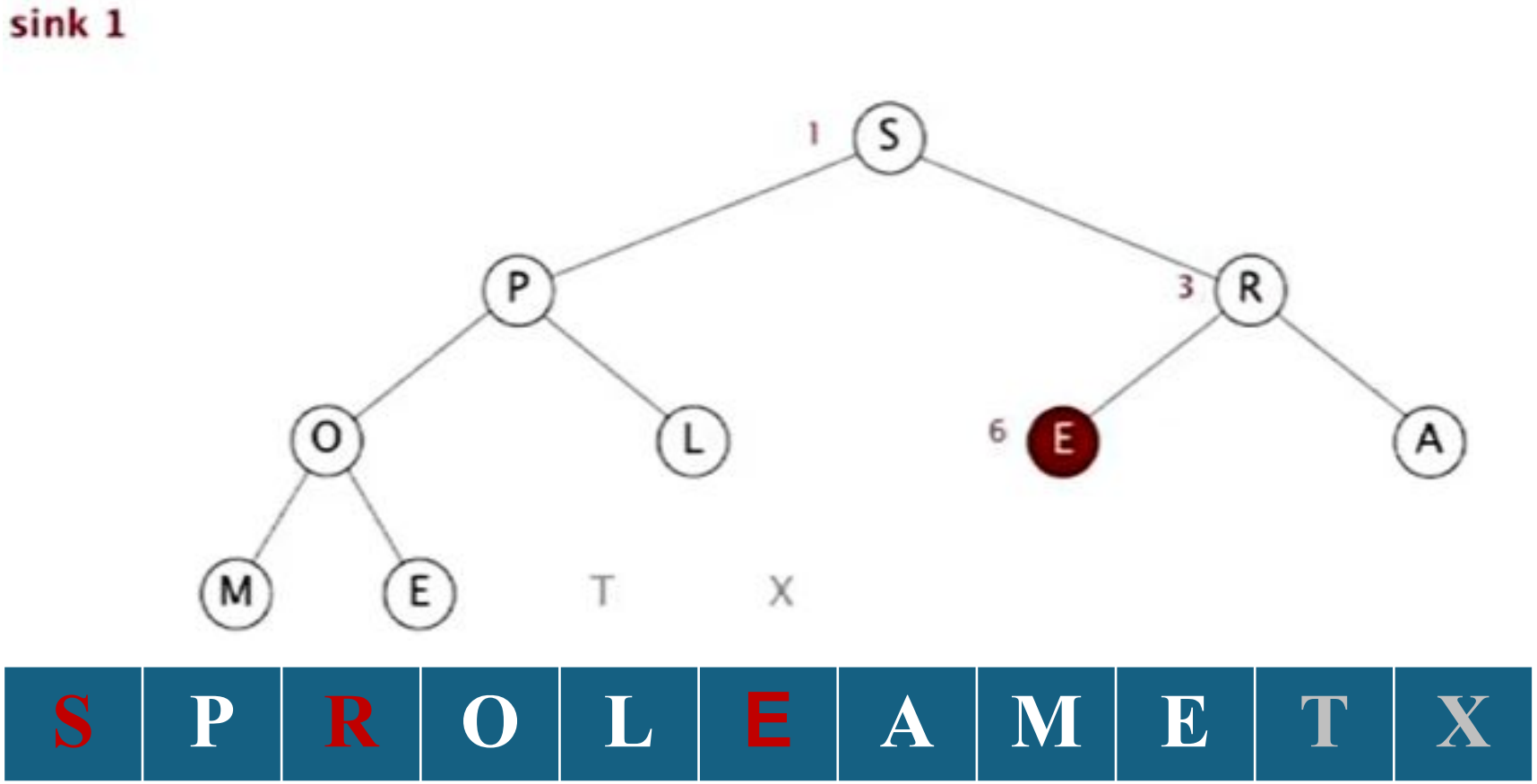
Heapsort demo

- Sortdown: Repeatedly delete the largest remaining item



Heapsort demo

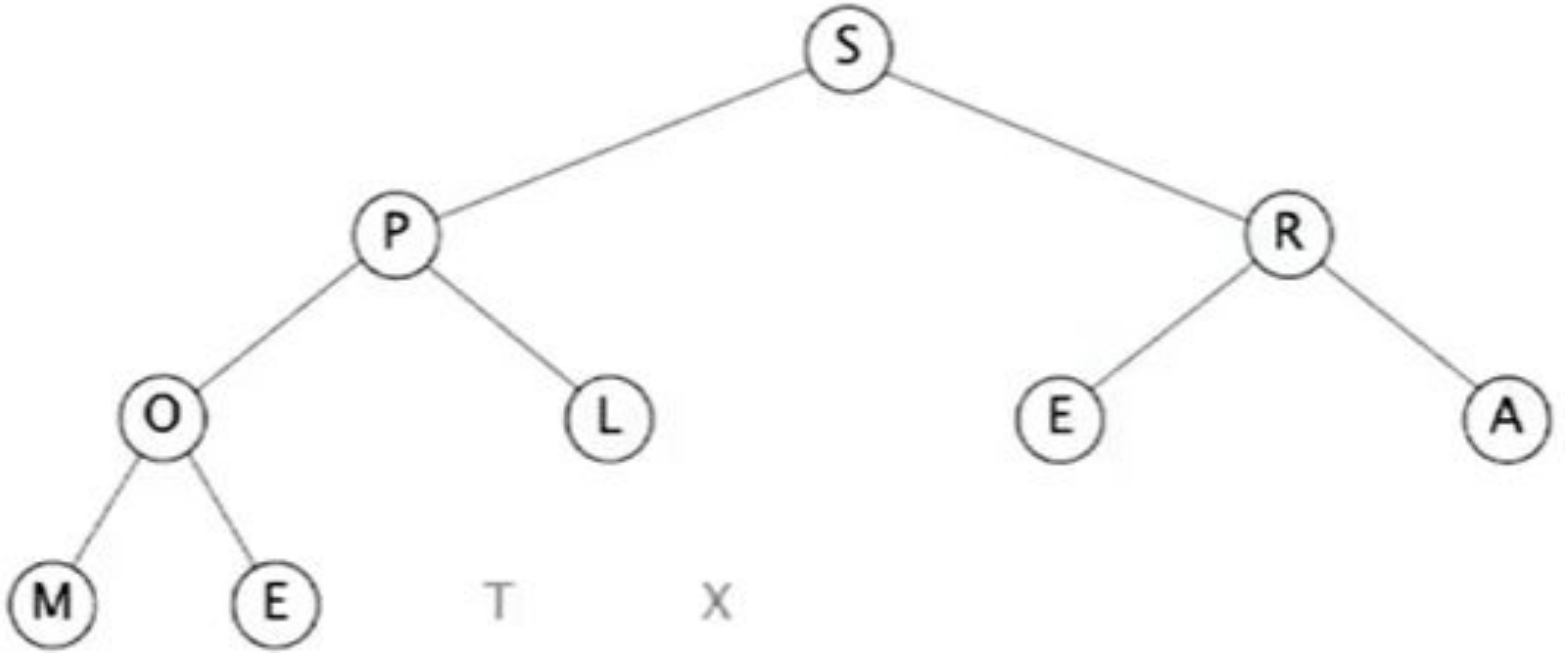
- Sortdown: Repeatedly delete the largest remaining item



S	P	R	O	L	E	A	M	E	T	X

Heapsort demo

- Sortdown: Repeatedly delete the largest remaining item



S	P	R	O	L	E	A	M	E	T	X

Heapsort demo

- Sortdown: Repeatedly delete the largest remaining item